[Week 11]

chelseey · 어제

통계 수정 삭제

Euron

Word2Vec

Word2Vec은 분포 가설을 이용해 각 단어를 고차원 벡터로 매핑

• 분포 가설 (Distributional Hypothesis)
: 같은 문맥에서 자주 함께 등장하는 단어는 의미가 비슷하다는 가설

ex.

- 내일 **자동차**를 타고 부산에 간다
- 내일 **비행기**를 타고 부산에 간다

→ 자동차와 비행기 주변에 나타나는 단어들이 비슷하므로, 둘은 의미상 가까움

• 분산 표현 (Distributed Representation)
: 단어를 고차원 벡터 공간에 매핑하여 단어의 의미를 담는 것
→ 유사한 문맥에서 등장하는 단어일수록 벡터 공간에서도 가깝게 위치

단어 벡터화

: 희소 표현, 밀집 표현

희소 표현 (Sparse Representation)

- 원-핫 인코딩, TF-IDF 등 빈도 기반 방법
- 벡터 차원이 '단어 사전 크기' 만큼 매우 커짐
- 대부분의 값은 0 → 메모리 낭비·계산 비효율 발생

표 6.3 단어의 희소 표현 (Sparse Representation)

단어	차원 0	차원 1	차원 2	차원 3	차원 4
소	0	1	0	0	0
잃고	1	0	0	0	0
외양간	0	0	0	1	0
고친다	0	0	0	0	1

밀집 표현 (Dense Representation)

- Word2Vec, GloVe, FastText 등의 임베딩 기법
- 고정된 실수 벡터
- 대부분 요소가 0이 아닌 실수 → 공간 효율적 사용
- 벡터 공간 거리로 단어 간 의미적 유사도 측정 가능

표 6.4 단어의 밀집 표현 (Dense Representation)

단어	차원 0	차원 1	...	차원 3	차원 4
소	0.3914	–0.1749	...	0.5912	0.1321
잃고	–0.2893	0.3814	...	–0.1492	–0.2814
외양간	0.4812	0.1214	...	–0.2745	0.0132
고친다	–0.1314	–0.2809	...	0.2014	0.3016

WordzVec은 밀집 표현을 위해 CBoW 와 Skip-gram 두 가지 방법을 사용

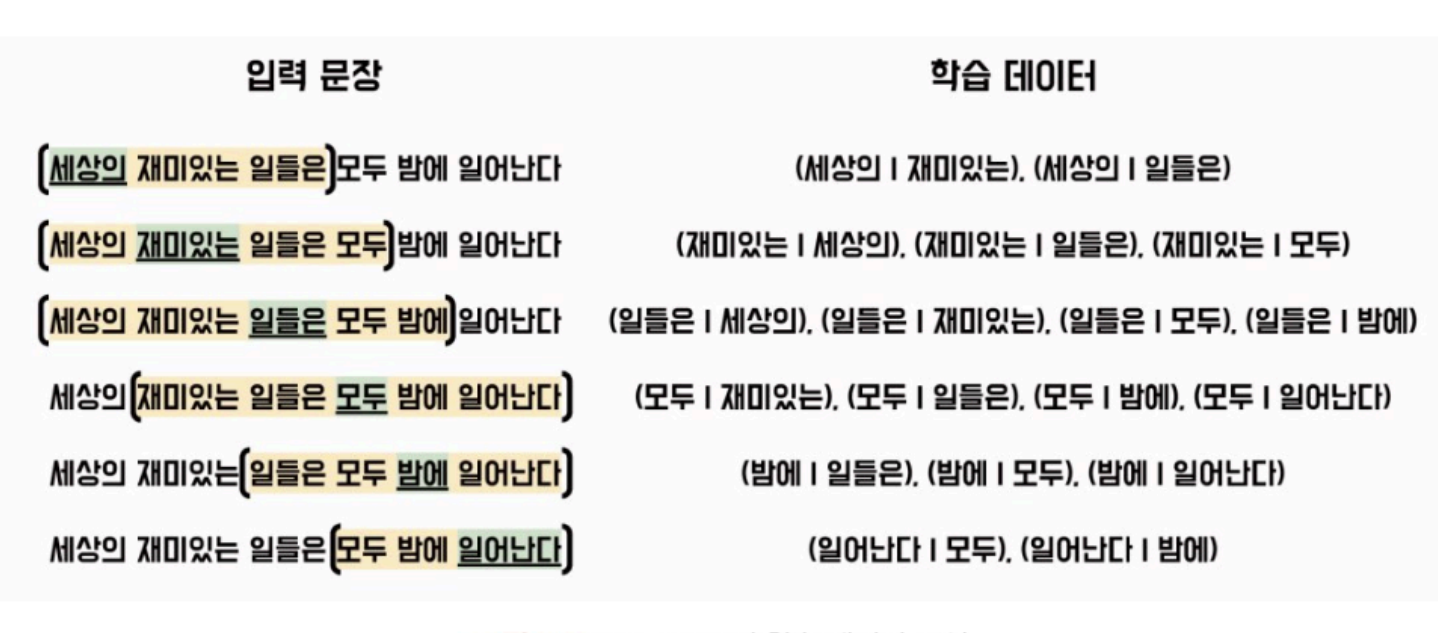
CBoW

: 주변 단어들(context words)을 보고
가운데 단어(center word)를 예측하는 방법

• 윈도우(Window)
학습 시 고려할 주변 단어의 범위

• 슬라이딩 윈도우
문장 전체에 걸쳐 윈도우를 한 칸씩 밀면서
(center word를 바꿔 가며) 학습하는 방식

ex. 학습 데이터 구성 (윈도우 크기 = 2)
: (주변 단어 | 중심 단어)



입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(세상의 | 재미있는), (세상의 | 일들은)
(세상의 재미있는 일들은 모두)밤에 일어난다	(재미있는 | 세상의), (재미있는 | 일들은), (재미있는 | 모두)
(세상의 재미있는 일들은 모두 밤에)일어난다	(일들은 | 세상의), (일들은 | 재미있는), (일들은 | 모두), (일들은 | 밤에)
세상의(재미있는 일들은 모두 밤에 일어난다)	(모두 | 재미있는), (모두 | 일들은), (모두 | 밤에), (모두 | 일어난다)
세상의 재미있는(일들은 모두 밤에 일어난다)	(밤에 | 일들은), (밤에 | 모두), (밤에 | 일어난다)
세상의 재미있는 일들은(모두 밤에 일어난다)	(일어난다 | 모두), (일어난다 | 밤에)

그림 6.6 Skip-gram의 학습 데이터 구성

• 모델 구조

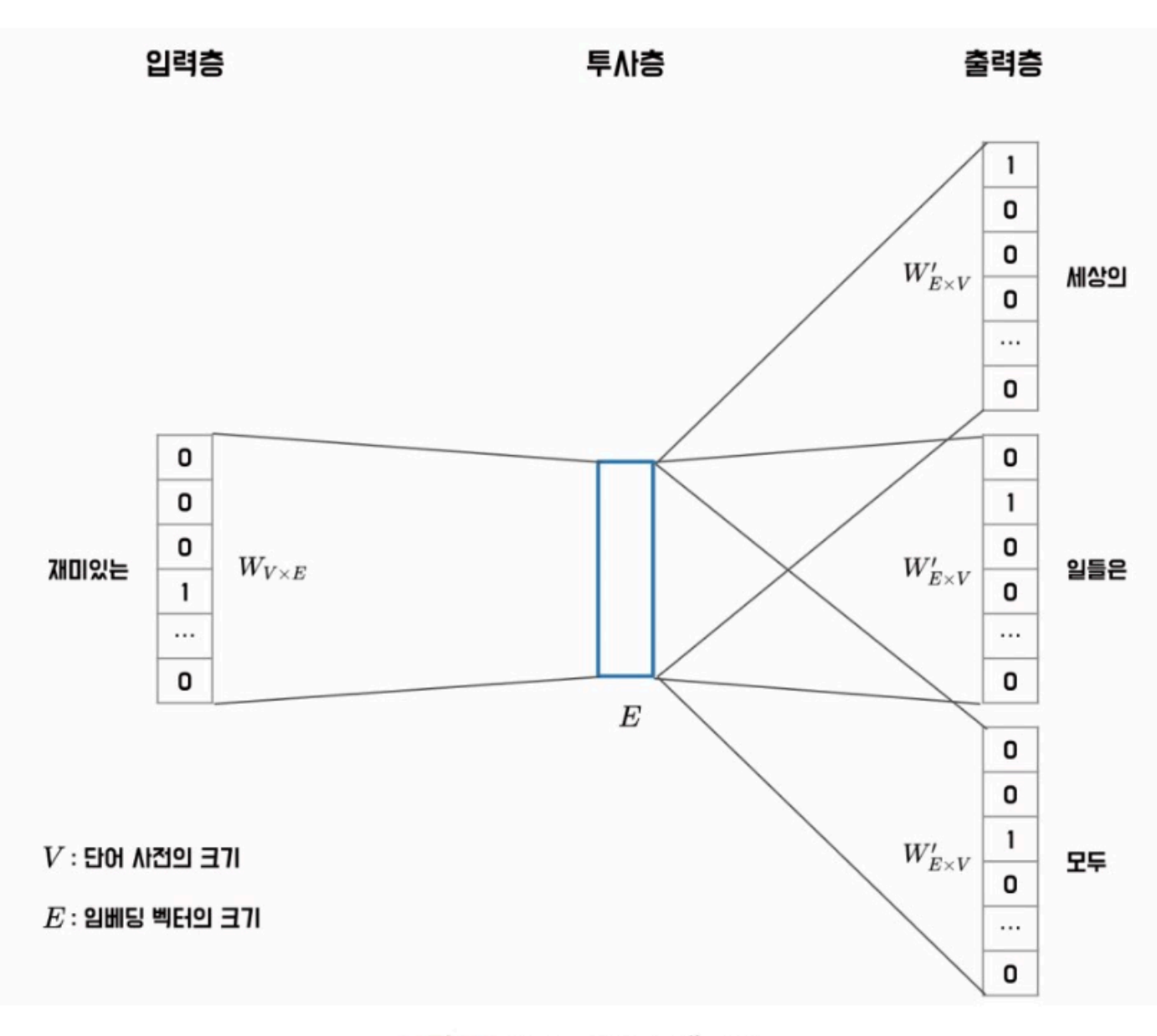


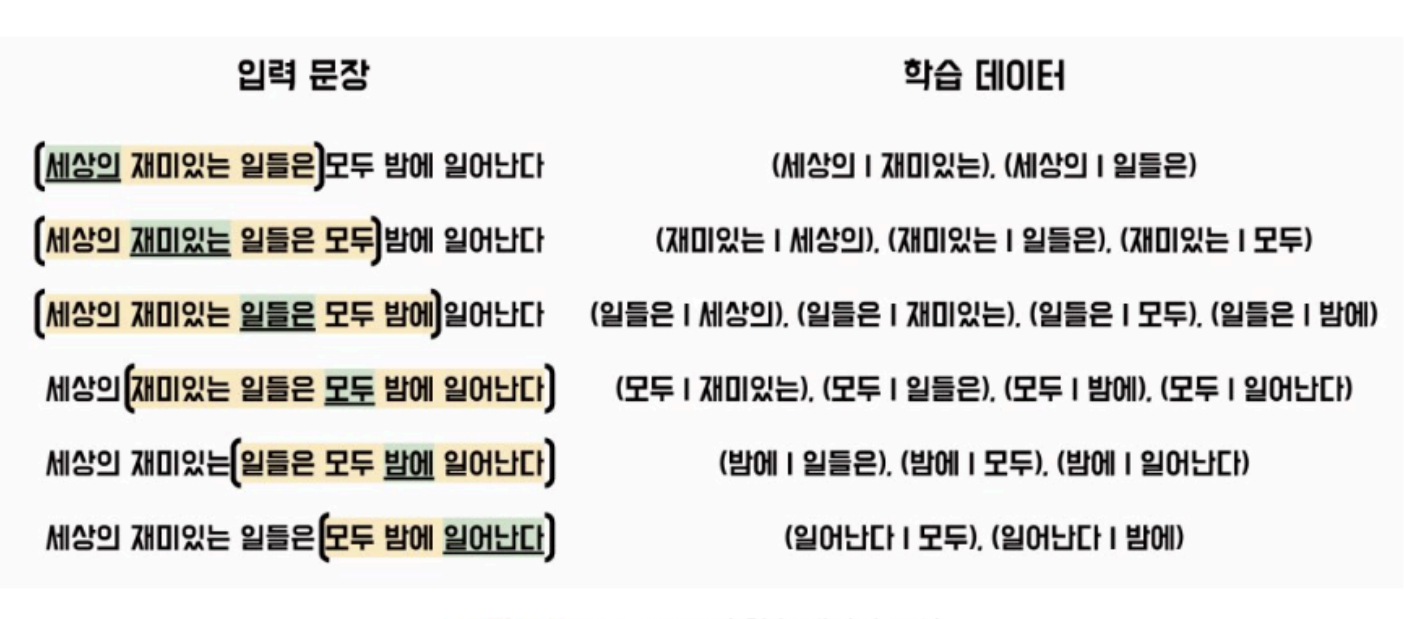
그림 6.7 Skip-gram 모델 구조

- 입력층 : 주변 단어들을 원-핫(one-hot) 벡터로 변환하여 입력
- 투사층(Projection Layer) : 각 원-핫 벡터를 크기 E의 임베딩 벡터로 변환
- 평균화 : 주변 단어 임베딩들의 평균을 구해 하나의 벡터로 결합
- 출력층 : 평균 벡터에 가중치 행렬(V×E 전치)을 곱해 차원 V(어휘 크기)짜리 출력 벡터를 만들고, 소프트맥스 활성화로 중심 단어를 예측

Skip-gram

: CBOW와 반대로 중심 단어(center word)를 입력으로 받아,
그 주변의 문맥 단어(context words)를 예측하는 방식

ex. 학습 데이터 구성 (윈도우 크기 = 2)



입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(세상의 | 재미있는), (세상의 | 일들은)
(세상의 재미있는 일들은 모두)밤에 일어난다	(재미있는 | 세상의), (재미있는 | 일들은), (재미있는 | 모두)
(세상의 재미있는 일들은 모두 밤에)일어난다	(일들은 | 세상의), (일들은 | 재미있는), (일들은 | 모두), (일들은 | 밤에)
세상의(재미있는 일들은 모두 밤에 일어난다)	(모두 | 재미있는), (모두 | 일들은), (모두 | 밤에), (모두 | 일어난다)
세상의 재미있는(일들은 모두 밤에 일어난다)	(밤에 | 일들은), (밤에 | 모두), (밤에 | 일어난다)
세상의 재미있는 일들은(모두 밤에 일어난다)	(일어난다 | 모두), (일어난다 | 밤에)

그림 6.6 Skip-gram의 학습 데이터 구성

• 모델 구조

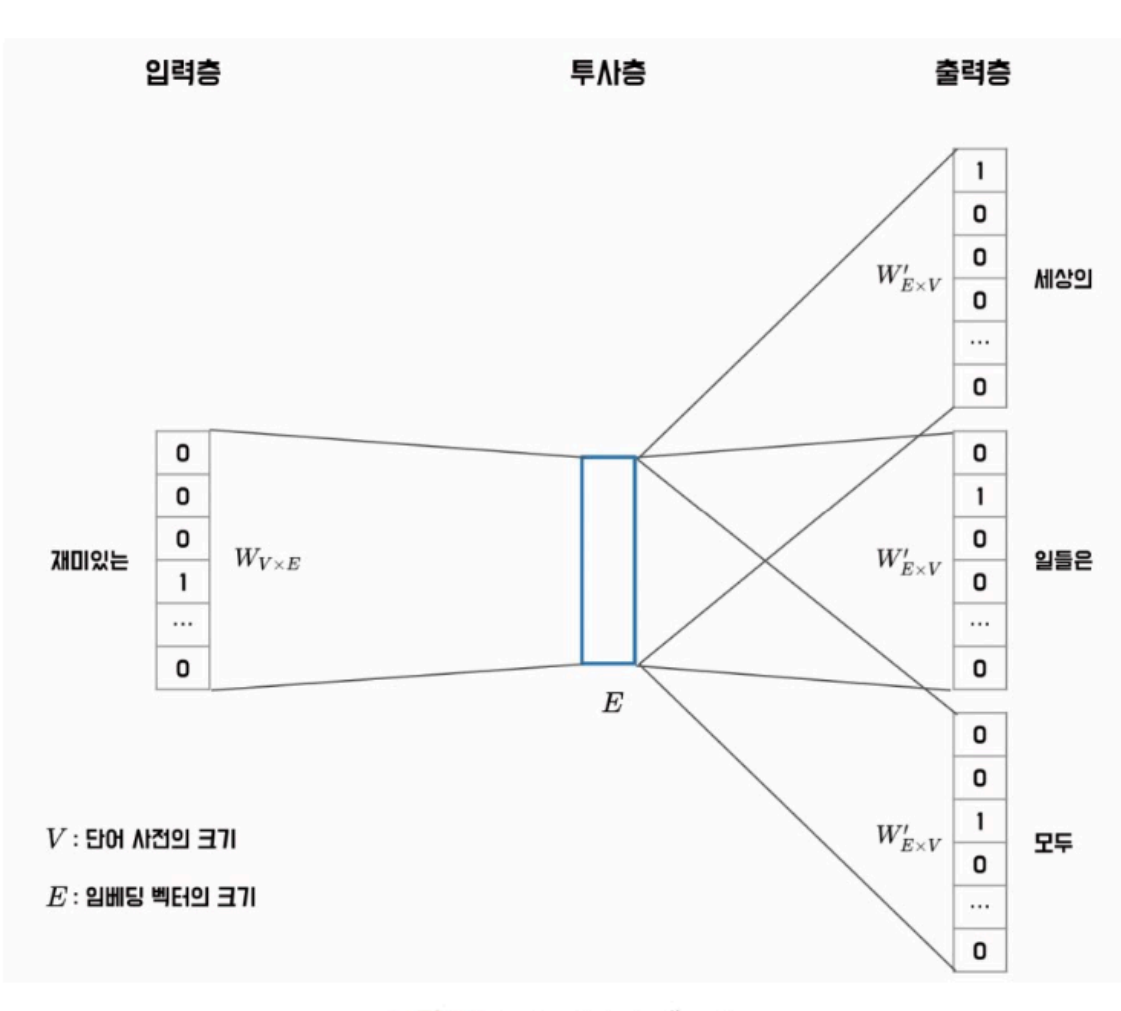


그림 6.7 Skip-gram 모델 구조

- 입력층 : 중심 단어를 원-핫(one-hot) 벡터로 표현
- 투사층(Projection Layer) : 원-핫 인덱스를 룩업 테이블로 변환해 크기 E의 임베딩 벡터를 추출
- 출력층 : 중심 단어 임베딩에 학습 가능한 가중치 행렬(크기 E×V 전치)을 곱해 어휘 크기 V짜리 로짓(logit) 벡터를 만들고,
  소프트맥스로 주변 단어를 예측

• 단점
출력층에 어휘 전체 대상 소프트맥스 계산이 필요해,
말뭉치 크기가 커지면 학습 속도가 느려짐
→ 계층적 소프트맥스, 네거티브 샘플링 기법 적용

계층적 소프트맥스

- 일반 소프트맥스 : 어휘(V) 전체에 대한 확률을 계산하느라 계산량이 많음.
- 계층적 소프트맥스 : 어휘를 이진 트리 구조로 표현하여 경로 상 노드만 순차적으로 계산.

• 트리 구조
자주 등장하는 단어일수록 트리 상단(루트에 가까운 위치)에 배치하여
평균 경로 길이를 줄임.

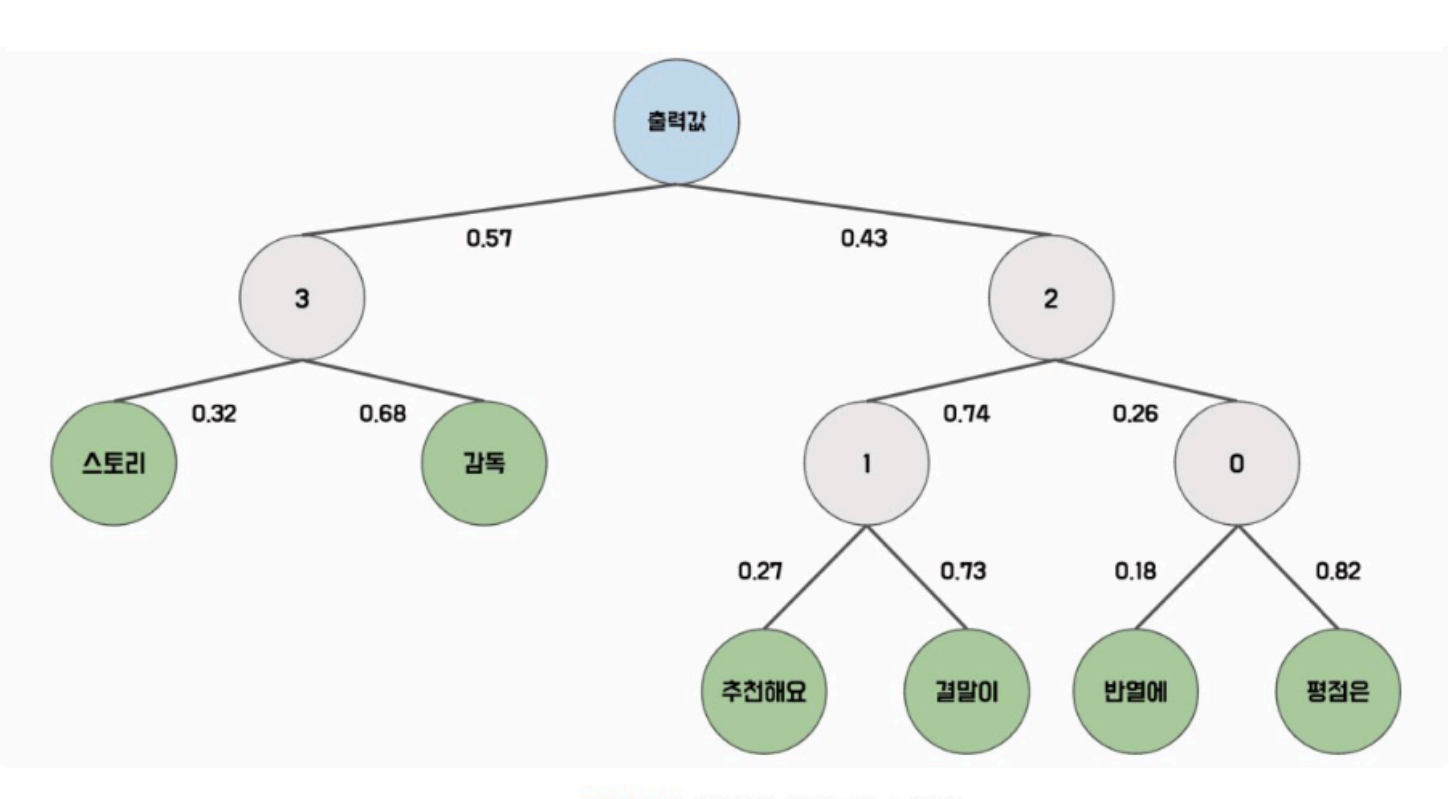


그림 6.8 계층적 소프트맥스 구조

- 내부 노드(internal node): 학습 가능한 벡터를 가짐.
- 리프 노드(leaf node) : 실제 예측 대상인 단어. 각 단어에 대응.

연산 복잡도: $O(log_2 V)$

네거티브 샘플링

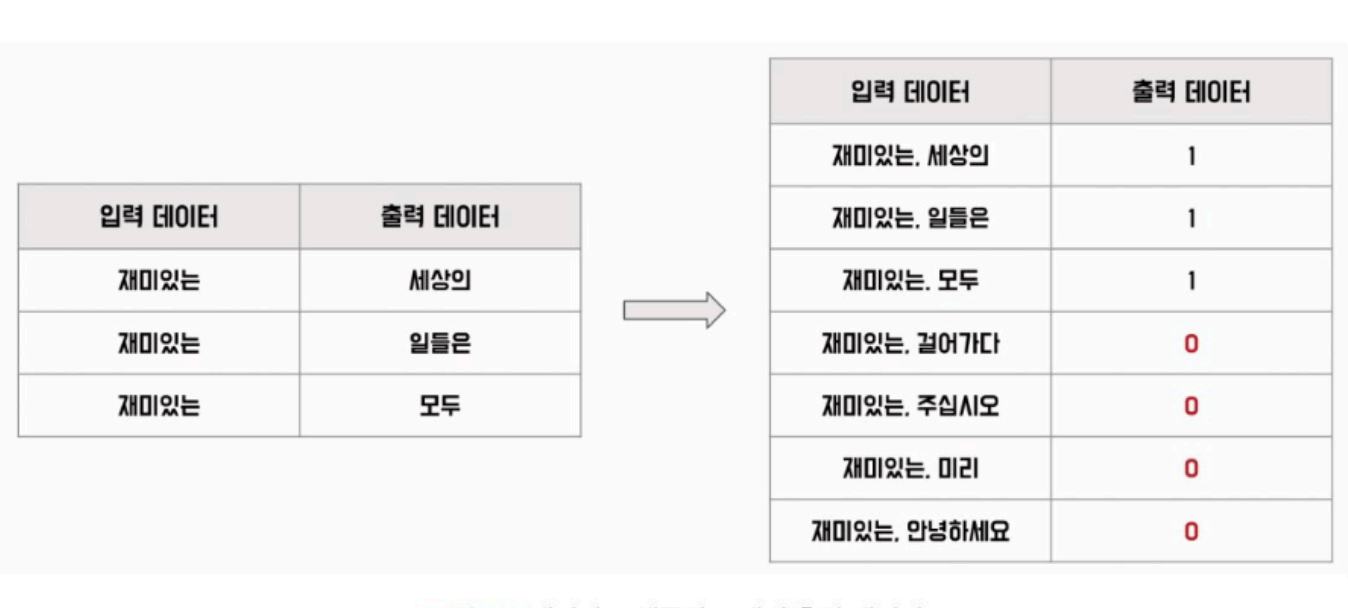
실제 문맥(positive) 단어 쌍 외에,
문맥에 등장하지 않는 임의의 단어(n개)를 "negative"로 샘플링
긍정 사례(positive)에는 레이블 1, 부정 사례(negative)에는 레이블 0을 부여해 이진 분류 문제로 학습

• 샘플링 확률
각 단어 w_i가 negative 샘플로 선택될 확률

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=1}^{V} f(w_j)^{0.75}}$$

$f(w_i)$: 말뭉치에서 단어 w_i의 등장 빈도
지수 0.75 : 실험적으로 최적화된 값

• 학습 절차



입력 데이터	출력 데이터
재미있는	세상의
재미있는	일들은
재미있는	모두

입력 데이터	출력 데이터
재미있는, 세상의	1
재미있는, 일들은	1
재미있는, 모두	1
재미있는, 걸어가다	0
재미있는, 주십시오	0
재미있는, 미리	0
재미있는, 안녕하세요	0

그림 6.9 네거티브 샘플링 모델의 훈련 데이터

- positive 쌍: 중심 단어–문맥 단어
- negative 쌍: 중심 단어–랜덤 샘플 단어(n개)
- 이진 분류(로지스틱 회귀)로 '이 쌍이 실제 문맥인가?' 판단
- positive 쌍은 시그모이드 출력이 1에 가깝도록,
  negative 쌍은 0에 가깝도록 가중치 업데이트

모델 실습 : Skip-gram

• Word2Vec 학습
전체 학습 대상 단어 수를 V, 임베딩 벡터 차원을 B로 설정
두 개의 가중치 행렬 $W_{i\to v}$ (입력→임베딩)과 $W'_{e\times v}$ (임베딩→출력)을
학습하며 최적화

• 룩업(lookup) 연산
: 이산적인 단어 ID(정수)를 연속적인 실수 벡터로 변환하는 과정
Word2Vec의 $W_{i\to v}$ 행렬에서 단어 ID에 해당하는 행(벡터)만 가져오는 연산

• `torch.nn.Embedding`
: 정수 인덱스(단어 ID)를 연속적인 실수 벡터(임베딩)로 변환하는 lookup 계층

```
embedding = torch.nn. Embedding(
num_embeddings,
embedding_dim,
padding_idx=None,
max_norm=None,
norm_type=2.0
)
```

• 기본 Skip-gram 모델 (`VanillaSkipgram`)

```
from torch import nn

class VanillaSkipgram(nn.Module):
    def init__ (self, vocab_size, embedding_dim):
        super().__init__()
        self.embedding = nn.Embedding(
        num embeddings= vocab_ size,
        embedding_dim=embedding_dim
    )
    self. linear = nn. Linear (
        in_features=embedding_dim,
        out_ features=vocab_size
    )

def forward(self, input_ids):
    embeddings = self.embedding(input_ids)
    output = self. linear (embeddings)
    return output
```

- 임베딩 계층 (`nn.Embedding(vocab_size, embedding_dim)`)
  중심 단어 ID → 고정 길이 벡터
- 출력 계층 (`nn.Linear(embedding_dim, vocab_size)`)
  투사된 중심 단어 벡터 → 모든 단어에 대한 점수(logit)

• 파이프라인

1. 데이터 로드 & 토크나이징
2. 단어 사전(vocabulary) 구축
   : 토큰별 출현 빈도를 집계하여, 상위 n_vocab개 토큰을 뽑아 단어 사전에 추가. 사전에 없는 토큰은 모두 `<unk>` 로 대체
3. ID ↔ 토큰 매핑
4. 학습용 단어 쌍 생성
   : `(center_word, context_word)` 쌍을 모두 수집
5. 토큰 → 인덱스 변환
   : 위에서 생성된 단어 쌍을 받아
   각 단어를 사전에 대응되는 정수 ID로 바꾼 뒤
   `[[center_id, context_id], …]` 형태의 리스트를 반환
6. 텐서 데이터셋 & 데이터로더 준비
   : 리스트를 [N,2] 형태의 torch.tensor로 변환한 뒤,
   배치 단위로 모델에 공급할 준비
7. 모델 & 손실함수 설정
8. 임베딩 행렬 추출
9. 코사인 유사도로 전체 사전과 비교

$$\text{cosine\_similarity}(a, b) = \frac{a \cdot b}{\|a\|\|b\|}$$

   : 한 단어 벡터 $a \in R^E$ 와 사전의 모든 벡터 $B \in R^{V\times E}$ 유사도 비교
10. 가장 유사한 상위 N개 단어 추출

모델 실습 : Gensim

Gensim 라이브러리를 이용해
Word2Vec 모델을 빠르게 학습하고 저장·불러오며, 학습된 벡터를 조회

• Word2Vec 객체 생성

```
from gensim.models import Word2Vec

model = Word2Vec(
    sentences=tokens,       # 토큰화된 문장들의 리스트
    vector_size=128,        # 임베딩 차원 수
    window=5,               # 컨텍스트 윈도우 크기
    min_count=1,            # 최소 단어 빈도(미등장 단어 무시)
    sg=1,                   # 1: Skip-gram, 0: CBOW
    hs=0,                   # 1: 계층적 소프트맥스 사용
    negative=5,             # 네거티브 샘플링 시 샘플 개수
    epochs=3,               # 학습 에폭 수
    max_final_vocab=10000,  # 최종 단어 사전 크기 상한
    workers=3               # 병렬 학습 스레드 수
)
```

• 모델 학습 및 저장/불러오기

```
# 학습
model = Word2Vec(...)

# 모델 저장
model.save("word2vec.model")

# 저장된 모델 불러오기
model = Word2Vec.load("word2vec.model")
```

Gensim은 `.save() / .load()` 메서드로 간편하게 모델 파일 관리 가능

• 학습된 임베딩 조회

```
# 단어의 벡터 가져오기
vec = model.wv["연기"]        # shape == (128,)

# 두 단어 간 유사도 계산
sim = model.wv.similarity("연기", "연기력")

# 가장 유사한 N개 단어 추출
top5 = model.wv.most_similar("연기", topn=5)
```

• 장단점

장점

- Cython/C++ 구현으로 대규모 말뭉치도 빠르게 학습
- 네거티브 샘플링, 계층적 소프트맥스 등 하이퍼파라미터 조정 용이

단점

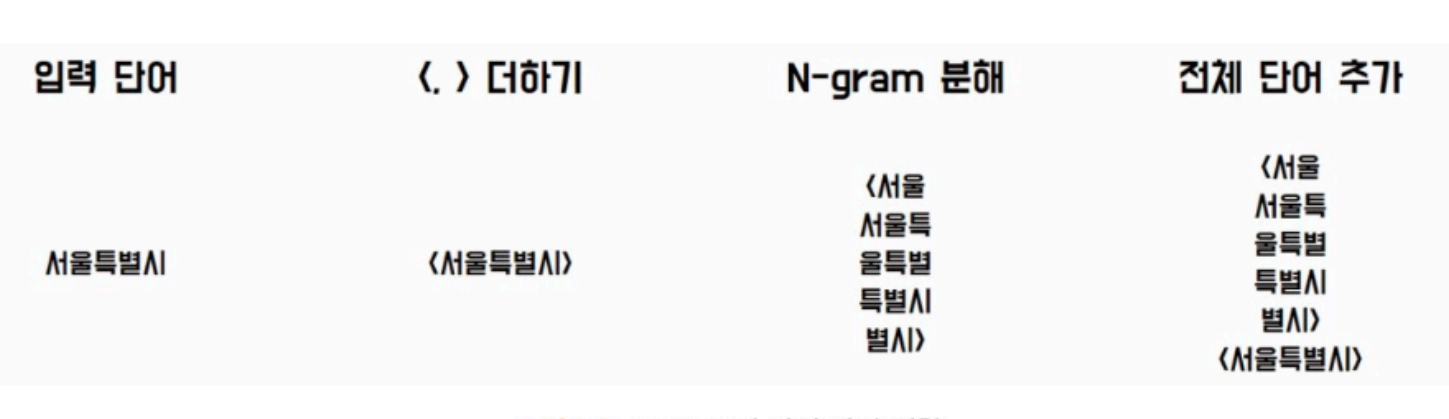
- 분포 가설 기반으로 학습하므로 단어의
  형태소적·구조적 특징(ex. 조사·어근 등)은 반영 어려움
- 한국어처럼 교착어 특성을 가진 언어에서는 OOV(사전 미등장)
  단어가 많아질 수 있음

fastText

텍스트 분류 및 텍스트 마이닝을 위한 알고리즘

• 하위 단어(subword) 기반 임베딩
어휘 단위(토큰)를 N-gram 단위로 분해한 뒤,
각 N-gram별 임베딩 벡터를 합산하여 원래 단어의 최종 벡터를 계산

ex. 3-gram 분해



입력 단어	⟨, ⟩ 더하기	N-gram 분해	전체 단어 추가
서울특별시	⟨서울특별시⟩	⟨서울 서울특 울특별 특별시 별시⟩	⟨서울 서울특 울특별 특별시 별시⟩ ⟨서울특별시⟩

그림 6.11 fastText의 하위 단어 집합

: <서울, 서울특, 울특별, 특별시, 별시>, plus 원본 토큰 <서울특별시>

• Word2Vec과의 차별점

- Word2Vec은 오직 토큰 단위 원-핫 → 임베딩 룩업만 수행
- fastText는 토큰뿐 아니라 토큰을 이루는 문자 n-gram까지 학습에 활용
  → 형태소적·어근적 정보 반영

• 처리 절차

1. 토큰 감싸기
   각 단어 앞뒤에 특별 기호(< , >)를 추가해 경계를 표시

2. n-gram 분해
   지정한 n-gram 길이로 문자 단위 N-gram 생성

3. 하위 단어 집합 구성
   분해된 각 N-gram과, 분해되지 않은 원토큰 자체를
   모두 하나의 하위 단어로 취급

4. 벡터 합산
   각 하위 단어별로 학습된 임베딩을 더해서 최종 단어 벡터 생성

모델 실습

FastText vs. Word2Vec

공통점

- 분산 표현 학습을 통해 단어를 고정 길이 벡터로 변환
- CBOW 또는 Skip-gram 구조 + 네거티브 샘플링(Negative Sampling)을 사용해 주변 맥락으로부터 단어 의미 추출
- 학습 후 비슷한 의미의 단어들은 벡터 공간 상에서도 가깝게 위치

차이점 : 단어 단위 vs. 하위단어(subword) 단위

Word2Vec

- 단어 전체(word) 를 기본 학습 단위로 삼음
- 사전에 없는(OOV) 단어는 임베딩 생성이 불가능하거나 `<unk>` 로 대체

FastText

- 하위단어(n-gram) 단위로 입력을 분해해 학습
- 각 n-gram 임베딩을 합산 또는 평균해 최종 단어 임베딩 계산
  → OOV 단어도 하위단어가 사전에 있으면 임베딩 가능

파이프라인

• KorNLI 데이터세트 전처리

```
from Korpora import Korpora
corpus = Korpora.load("kornli")

# KorNLI: 한국어 자연어 추론(NLI) 데이터셋
texts = corpus.get_all_texts()        # 단일 문장 리스트
pairs = corpus.get_all_pairs()        # (문장1, 문장2, 레이블) 튜플 리스트
corpus_texts = texts + [p[0] + " " + p[1] for p in pairs]
tokens = [sentence.split() for sentence in corpus_texts]
```

• fastText 모델
KorNLI 데이터로부터 전처리된 토큰(tokens)을 입력으로 FastText 모델을 학습하고 저장

```
from gensim.models import FastText

fastText = FastText(
    sentences=tokens,    # 학습할 문장: 토큰 리스트들의 리스트
    vector_size=128,     # 출력 임베딩 벡터 차원
    window=5,            # 컨텍스트 윈도우 크기
    min_count=5,         # 이 빈도 미만 단어는 무시
    sg=1,                # 1: Skip-gram, 0: CBOW
    epochs=3,            # 학습 에폭 수
    min_n=2,             # 생성할 n-gram의 최소 길이
    max_n=6              # 생성할 n-gram의 최대 길이
)
```

• fastText OOV 처리

```
oov_token = "사랑해요"     #OOV 단어 지정
oov_vector = fastText.wv[oov_token] #임베딩 추출
print(oov_token in fastText.wv.index_to_key)  # 사전 등재 여부 확인
print(fastText.wv.most_similar(oov_vector, topn=5)) # 유사 단어 검색
```

순환 신경망

: 순서가 있는 연속 데이터(Sequence data)를 처리하기에 적합한 신경망 구조
시계열 데이터, 자연어 문장 등
이전 시점의 정보가 현재 처리에 영향을 미치는 경우 효과적

핵심 아이디어

- 각 시점(time step)에서의 출력을 다음 시점의 입력으로 사용하여,
  순차적 의존성(Dependency)을 모델링
- 내부에 은닉 상태(hidden state)를 유지해 과거 정보를 누적

연속 데이터의 특성

• 과거 영향력
ex. 주가 데이터에서 3월 7일의 가격은 3월 6일 가격의 영향을 받음
문장에서도 단어들이 앞선 단어들의 의미를 이어받아 문맥을 형성

• 상호작용(Interaction)
문장 내 단어들 사이에는 복잡한 상관관계(correlation)가 존재
ex. "금요일이 지나면 _" 다음에 등장할 단어는 "주말"이 될 가능성이 높음

순환 신경망

순차적(시간적) 연속 데이터(시계열, 자연어, 음성 등)를 처리하기 위해
고안된 신경망
각 시점(time step)의 입력과 이전 시점의 상태를 함께 사용해 현재 출력을 계산

셀(Cell) 구조

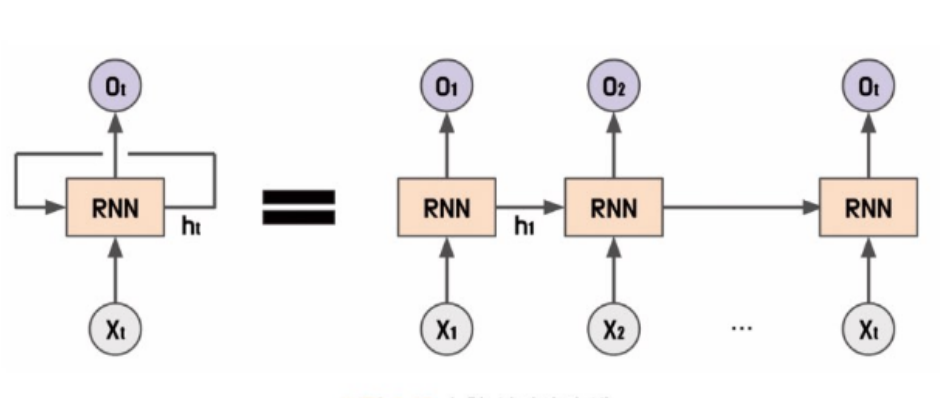


그림 6.12 순환 신경망의 셀

입력 x_t와 이전 은닉 상태(hidden state) h_{t-1}을 받아

• 은닉 상태 갱신

$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

W_{hh} : 이전 상태에 대한 가중치
W_{xh} : 현재 입력에 대한 가중치

W_{xh} : 현재 입력에 대한 가중치
b_h : 은닉 상태 편향
σ_h : 은닉 상태 활성화 함수 (ex.tanh, ReLU)

- 출력 계산

$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

W_{hy} : 은닉 상태 → 출력 가중치
b_y : 출력 편향
σ_y : 출력 활성화 함수 (ex. Softmax, Sigmoid)

순환 신경망의 구조 : 단순 순환 구조, 일대다 구조, 다대일 구조, 다대다 구조

일대다 구조

하나의 입력(시퀀스)으로부터 여러 개의 출력을 생성하는 순환 신경망 구조

입력은 하나의 시퀀스(문장, 이미지 특징 등),
출력은 그에 대응하는 여러 시점의 값들(단어, 캡션 등)

적용 예시

- 자연어 처리
  : 문장 전체(입력)를 받아 각 단어의 품사 태그 시퀀스를 출력
- 이미지 캡셔닝(Image Captioning)
  : 이미지 특징(입력)으로부터
  "A cat is sleeping" 같은 설명 문장(시퀀스)을 생성

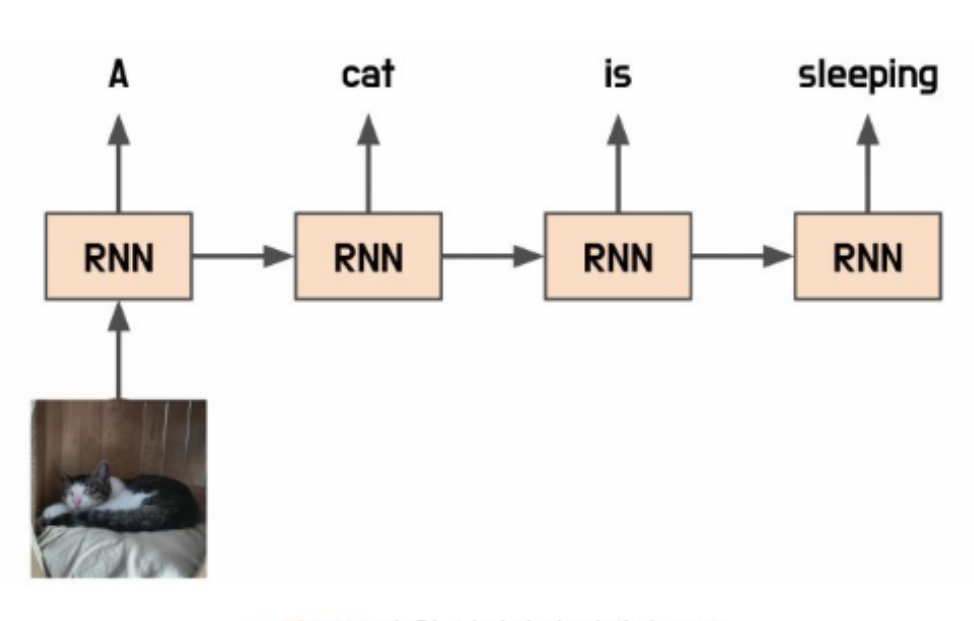


그림 6.13 순환 신경망의 일대다 구조

구현 시 고려 사항

- 출력 시퀀스 길이 예측
  : 출력할 단어 수나 문장 길이를 미리 알아야 함
  또는 종료 토큰(<EOS>)을 학습시켜 동적으로 마칠 수도 있음

다대일 구조

여러 시점의 입력 시퀀스 전체를 처리한 뒤,
하나의 출력값을 생성하는 순환 신경망 구조

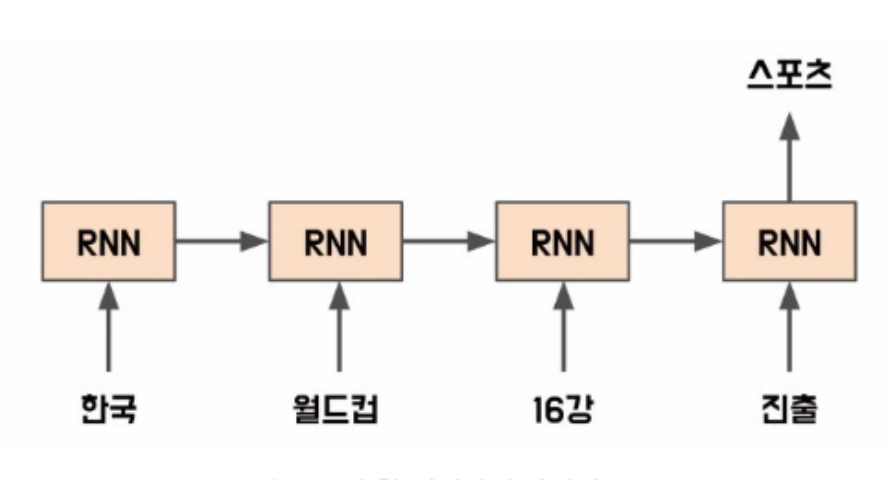


그림 6.14 순환 신경망의 다대일 구조

적용 예시

- 감성 분류(Sentiment Analysis)
  : 문장 전체를 입력받아 긍정/부정 레이블 하나를 출력
- 자연어 추론(NLI)
  : 두 문장의 관계(entailment, neutral, contradiction) 하나를 예측

다대다 구조

입력 시퀀스와 출력 시퀀스의 길이가 모두 여러 개인 경우에
사용하는 순환 신경망 구조

시퀀스-시퀀스 구조: Seq2Seq (인코더–디코더)

- 인코더(Encoder)
  입력 시퀀스 $(x_1, x_2, \ldots, x_T)$을 순환 처리
  최종 은닉 상태 또는 모든 은닉 상태를 고정 차원 벡터(컨텍스트 벡터)로 압축

- 디코더(Decoder)
  인코더의 컨텍스트 벡터를 초기 입력(및 초기 은닉 상태)으로 받아
  출력 시퀀스 $(y_1, y_2, \ldots, y_{T'})$를 한 시점씩 생성

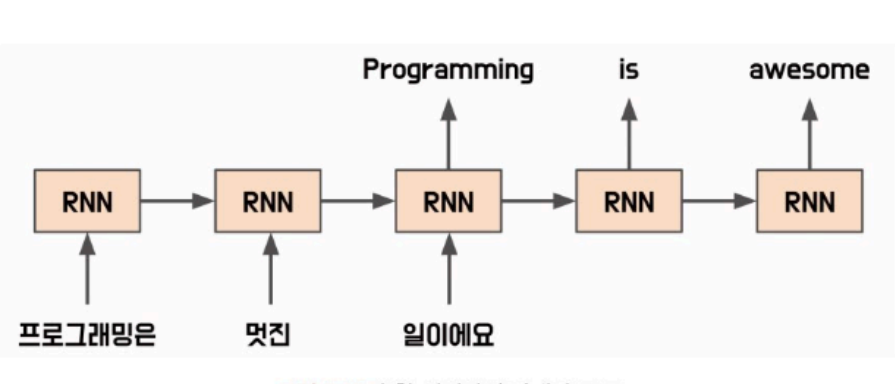


그림 6.15 순환 신경망의 다대다 구조

적용 예시

- 번역(Translation)
  : ex. 한국어 문장→ 영어 문장
- 음성 인식(Speech-to-Text)
  : 음성 신호(프레임 시퀀스) → 텍스트 토큰 시퀀스

양방향 순환 신경망

과거(이전 시점) 정보뿐 아니라 미래(이후 시점) 정보도 함께 활용해
각 시점의 출력을 계산하는 순환 신경망(RNN)의 확장 구조

일반 RNN은 현재 시점 t의 은닉 상태 h_t를 계산할 때
과거 시점 $t-1$의 정보만 사용.
하지만 자연어 문장에서는 뒤쪽 단어 정보가 앞쪽 단어 예측에 도움이 될 수 있음

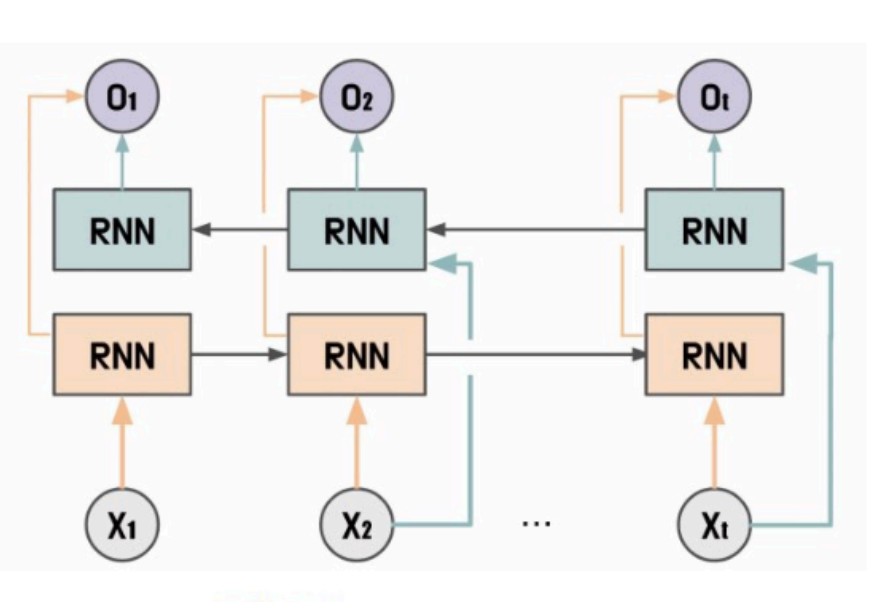


그림 6.16 양방향 순환 신경망 구조

구조

1. 정방향 RNN : 입력 시퀀스를 $x_1 \to x_2 \to \cdots \to x_T$ 순으로 처리하며
   은닉 상태 $\overrightarrow{h}_t$ 계산
2. 역방향 RNN : 동일한 입력을 거꾸로 $(x_T \to x_{T-1} \to \cdots \to x_1)$ 처리하며 은닉 상태 $\overleftarrow{h}_t$ 계산
3. 출력 결합: 각 시점 t에서 두 방향의 은닉 상태를 concatenate
   (또는 sum)하여 최종 은닉 표현

→ "양방향으로 읽기"를 통해 문맥의 앞뒤 관계를 모두 반영

다중 순환 신경망

여러 개의 RNN 층을 위아래로 쌓아 구성한 모델로,
각 층의 출력을 다음 층의 입력으로 전달하며 순차 데이터를 처리

구조

1. 첫 번째(하위) RNN 층이 원시 입력 시퀀스
   $(x_1, x_2 \ldots)$ 를 받아 은닉 상태와 출력을 계산
2. 두 번째 RNN 층은 첫 번째 층의 출력을 시퀀스 형태로 받아
   다시 은닉 상태·출력을 계산
3. 원하는 층 수만큼 쌓아 올려, 마지막 층의 출력을 최종 예측에 활용

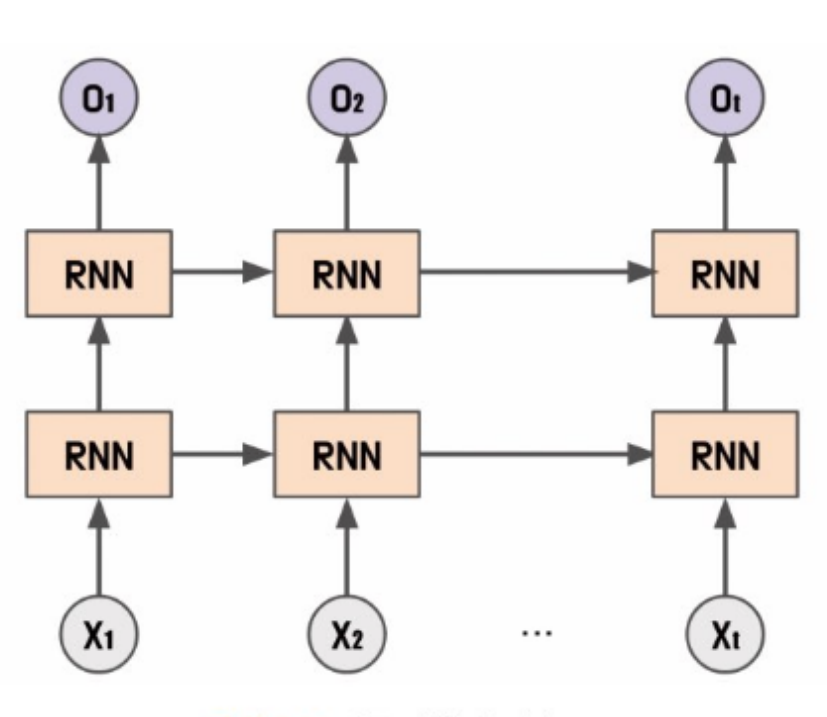


그림 6.17 다중 순환 신경망 구조

장점

- 표현력 강화 : 층이 깊어질수록 더 추상적이고 복잡한 패턴을 학습
- 특징 계층화 : 저층은 저수준(로컬) 특성, 고층은 고수준(글로벌) 특성을 추출

순환 신경망 클래스

- 순환 신경망 클래스

```
import torch.nn as nn

# nn.RNN 순환 신경망 클래스 생성
rnn = nn.RNN(
    input_size,          # 입력 벡터의 특징 차원 수 (예: 임베딩 차원)
    hidden_size,         # 은닉 상태 벡터의 크기
    num_layers=1,        # RNN 레이어(층) 개수. 1 이상이면 스택된 구조.
    nonlinearity="tanh", # 은닉 상태를 업데이트할 때 사용할 활성화 함수: "tanh" 또는 "relu"
    bias=False,          # 각 레이어의 편향(bias) 사용 여부
    batch_first=True,    # 입력 텐서 형태를 (batch, seq_len, feature)로 처리할지 여부
    dropout=0,           # 다중 레이어 RNN 사이에 적용할 드롭아웃 확률
    bidirectional=False  # 양방향 RNN 사용 여부. True면 순방향+역방향
)
```

- 양방향 다층 신경망 (ex. 3층 양방향 RNN)

```
import torch
from torch import nn

# 설정
input_size  = 128
hidden_size = 256
num_layers  = 3
bidirectional= True

# 모델 정의
model = nn.RNN(
    input_size=input_size,
    hidden_size=hidden_size,
    num_layers=num_layers,
    nonlinearity="tanh",
    batch_first=True,
    bidirectional=bidirectional
)

# 더미 입력 및 초기 은닉 상태
batch_size     = 4
sequence_len   = 6
inputs   = torch.randn(batch_size, sequence_len, input_size)
h_0      = torch.randn(num_layers * 2, batch_size, hidden_size)  # 양방향이므로 num_layers×2

# 순전파
outputs, hidden = model(inputs, h_0)

print(outputs.shape)  # torch.Size([4, 6, 512])  (512 = 2×256)
print(hidden.shape)   # torch.Size([6, 4, 256])   (6 = 3층×2방향)
```

장단기 메모리(LSTM)

기본 RNN이 가진 장기 의존성 문제와
기울기 소실/폭주 문제를 해결하기 위해 고안된 구조

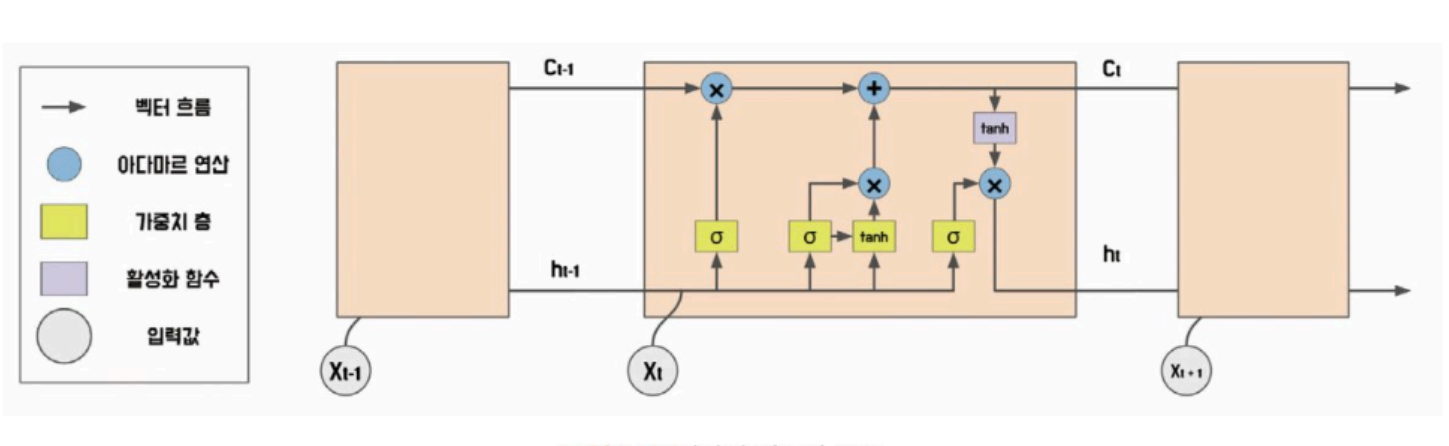


그림 6.18 장단기 메모리 구조

- 셀 상태(Cell state, C_t)
  장기간 정보를 보존하는 역할
  망각 게이트와 입력 게이트에 의해 제어됨

- 망각 게이트(Forget gate)
  이전 셀 상태(C_{t-1})에서 어떤 정보를 지울지 결정
  현재 입력 x_t와 이전 은닉 상태 h_{t-1}를 받아 σ(·)를 통해
  [0,1]의 비율로 얼마나 기억을 지울지 계산

- 입력 게이트(Input gate)
  현재 입력에서 어떤 새 정보를 셀 상태에 추가할지 결정
  x_t와 h_{t-1}를 받아 σ(·)로 얼마나 업데이트할지 비율 산출
  동시에 tanh를 통해 추가할 후보 정보 $\tilde{C}_t$ 생성

- 셀 상태 업데이트
  C_t = (망각 게이트 ⊙ C_{t-1}) + (입력 게이트 ⊙ $\tilde{C}_t$)
  이전 기억을 일부 지우고, 새 정보를 덧붙이는 방식

- 출력 게이트(Output gate)
  최종 은닉 상태 h_t(= 다음 시점 RNN 입력값 및 출력값) 생성 여부 결정

장단기 메모리 구조

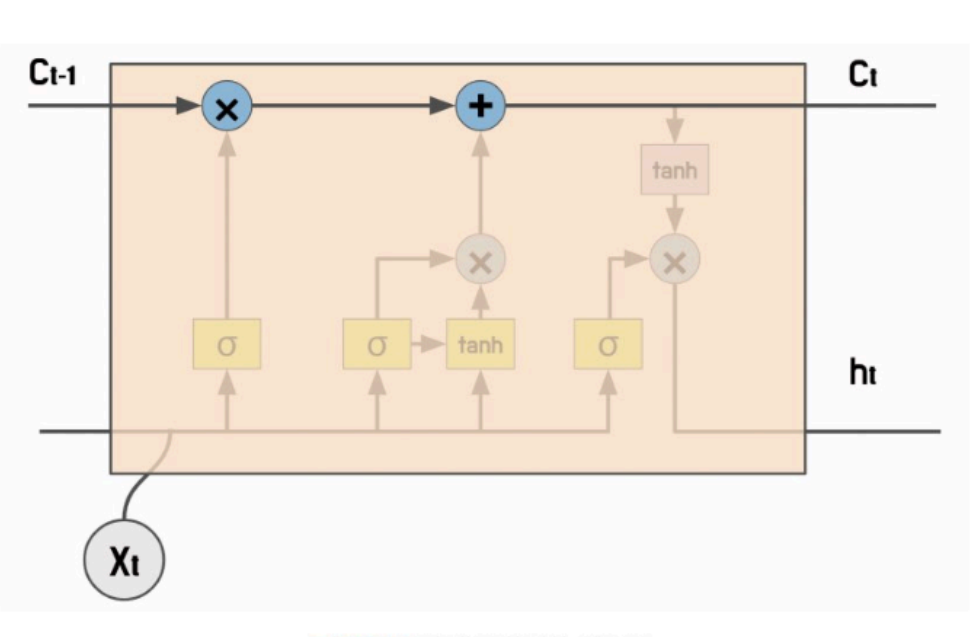


그림 6.19 장단기 메모리 셀 상태

- 망각 게이트 (Forget Gate)

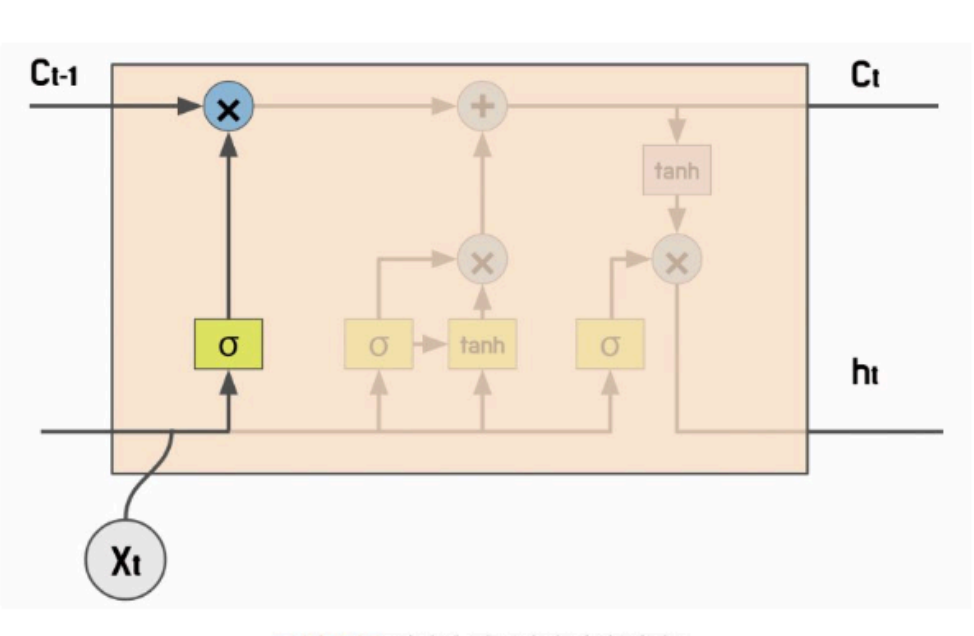


그림 6.20 장단기 메모리의 망각 게이트

이전 셀 상태 C_{t-1}에서 어떤 정보를 버릴지 결정

$$f_t = \sigma(W_x^{(f)}x_t + W_h^{(f)}h_{t-1} + b^{(f)})$$

$f_t \in (0,1)$로, 1에 가까울수록 이전 정보를 그대로 유지, 0에 가까울수록 삭제

- 입력 게이트 (Input Gate) & 후보 셀 상태

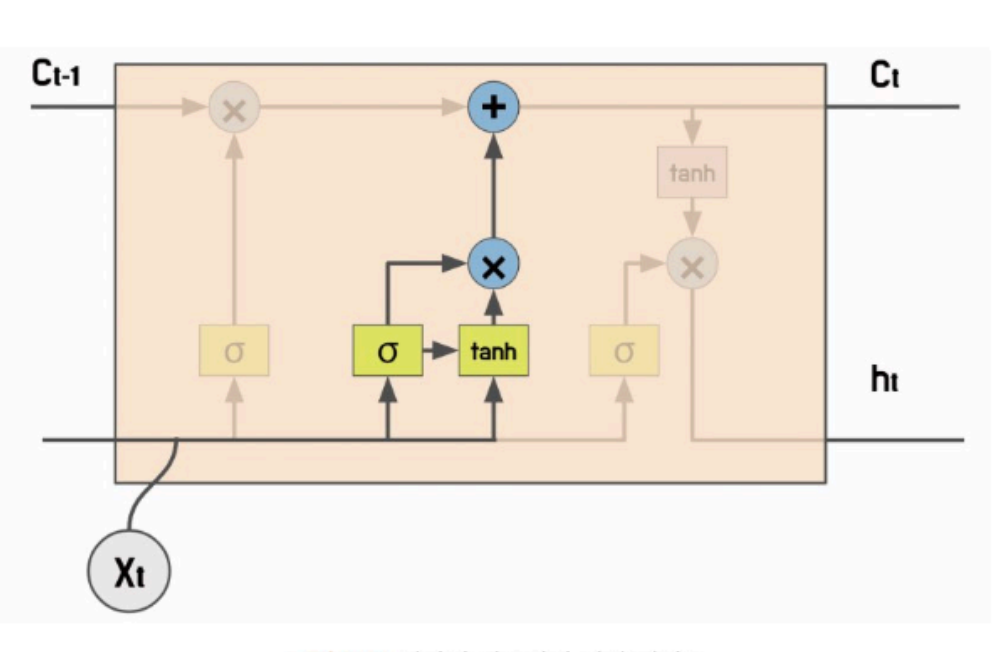


그림 6.21 장단기 메모리의 기억 게이트

새로 들어온 정보 중 얼마나 셀 상태에 추가할지, 어떤 새 정보(후보)를
생성할지 결정

$$i_t = \sigma(W_x^{(i)}x_t + W_h^{(i)}h_{t-1} + b^{(i)})$$
$$\tilde{C}_t = \tanh(W_x^{(c)}x_t + W_h^{(c)}h_{t-1} + b^{(c)})$$

$i_t \in (0,1)$: 얼마나 많이 업데이트할지 비율
$\tilde{C}_{tt} \in (-1,1)$: 추가될 후보 정보

- 셀 상태 업데이트

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

이전 기억은 f_t만큼 남기고, 새로운 정보는 i_t만큼 덧붙여 종합한 셀 상태

- 출력 게이트 (Output Gate)

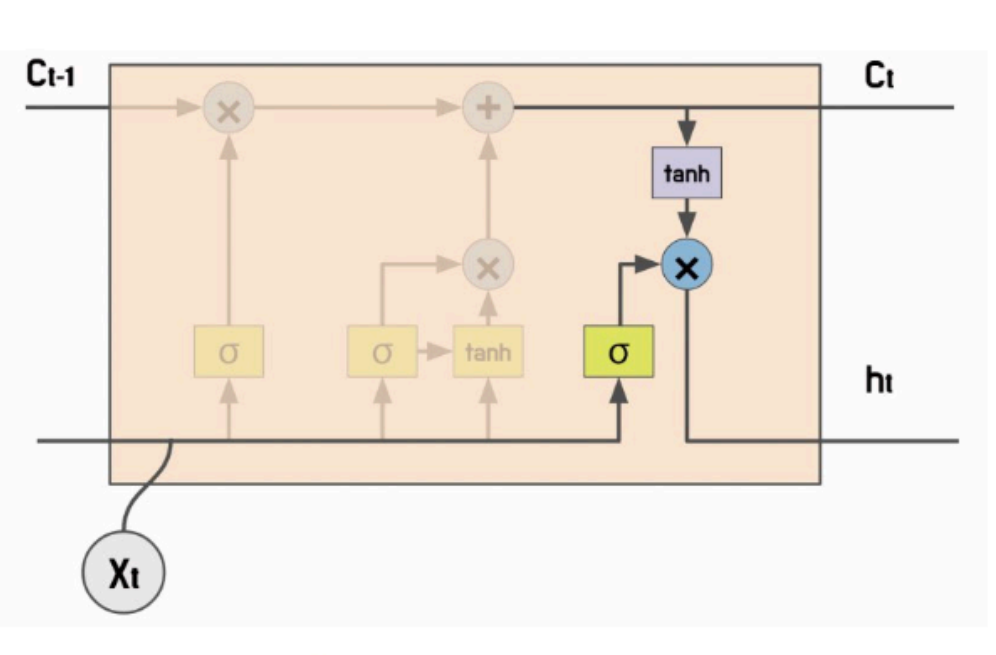


그림 6.22 장단기 메모리의 출력 게이트

최종 은닉 상태 h_t로 어떤 정보를 꺼낼지 결정

$$o_t = \sigma(W_x^{(o)}x_t + W_h^{(o)}h_{t-1} + b^{(o)})$$
$$h_t = o_t \odot \tanh(C_t)$$

$o_t \in (0,1)$만큼만 셀 상태를 출력값으로 내보냄

장단기 메모리 클래스

장단기 메모리 클래스

```
import torch.nn as nn

# 장단기 메모리(LSTM) 레이어 정의
lstm = nn.LSTM(
    input_size,       # 입력 특징 벡터의 크기 (예: 단어 임베딩 차원)
    hidden_size,      # 은닉 상태의 크기 (LSTM 레이어 출력 차원)
    num_layers=1,     # LSTM 레이어의 층 수 (2 이상이면 스택된 LSTM)
    bias=False,       # 편향(bias) 사용 여부 (False면 내부 연산 시 편향을 사용하지 않음)
    batch_first=True, # 입력 텐서 형식을 (batch, seq_len, feature)로 받을지 여부
    dropout=0,        # 여러 LSTM 층 사이의 드롭아웃 확률 (0이면 드롭아웃 미사용)
    bidirectional=False,  # 양방향 LSTM 여부 (True면 순방향+역방향 처리)
    proj_size=0           # 선형 투사(projection) 크기; 0이면 투사 생략, hidden_size 로 유지
)
```

장단기 메모리(LSTM) 주요 특징

- RNN과 유사한 구조
  시퀀스 데이터를 처리하기 위해 순환 구조를 갖고,
  이전 시점의 정보를 은닉 상태에 저장해 다음 시점으로 전달
- 사용 시 주의사항
  batch_first 설정에 따라 입력·출력 텐서의 차원 순서가 달라짐

모델 실습

ex. 문장 긍·부정 분류 모델 학습 파이프라인

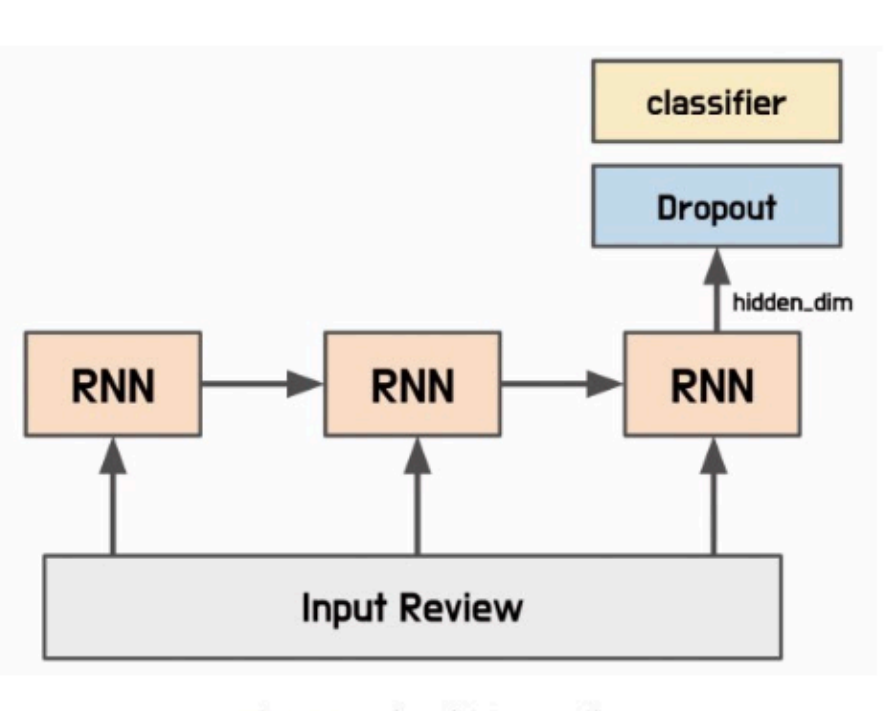


그림 6.23 긍/부정 분류 모델 구조

1. 임베딩 계층 (`nn.Embedding`)
   입력 단어를 실수 벡터로 바꿔주는 역할
   크기 = (어휘 사전 크기 `n_vocab`) × (임베딩 차원 `embedding_dim`)
   `padding_idx=0` 으로 패딩 토큰에 대응

2. 문장 분류 모델 (RNN 또는 LSTM)
   RNN일 때 :

```
self.model = nn.RNN(
    input_size=embedding_dim,
    hidden_size=hidden_dim,
    num_layers=n_layers,
    bidirectional=bidirectional,
    dropout=dropout,
    batch_first=True
)
```

3. 데이터 불러오기 & 분할

4. 토크나이저로 토큰화 & 단어 빈도 집계

5. 토큰 → 인덱스 맵 생성

6. 정수 인코딩
   : 각 리뷰의 토큰을 `token_to_id` 로 매핑. 사전에 없으면 `unk_id` 사용

7. 패딩/트렁케이팅

```
def pad_sequences(seqs, max_length, pad_value):
    result = []
    for seq in seqs:
        seq = seq[:max_length]                       # 너무 길면 자르고
        pad_len = max_length - len(seq)              # 남는 칸만큼
        result.append(seq + [pad_value]*pad_len)     # 뒤쪽에 <pad> 추가
    return np.asarray(result)

max_length = 32
train_ids = pad_sequences(train_ids, max_length, pad_id)
test_ids  = pad_sequences(test_ids,  max_length, pad_id)
```

8. PyTorch 텐서 & 데이터로더 변환

```
from torch.utils.data import TensorDataset, DataLoader

X_train = torch.tensor(train_ids)                          # [45000, 32]
y_train = torch.tensor(train.label.values, dtype=torch.float32)
X_test  = torch.tensor(test_ids)
y_test  = torch.tensor(test.label.values, dtype=torch.float32)

train_ds = TensorDataset(X_train, y_train)
test_ds  = TensorDataset(X_test,  y_test)

train_loader = DataLoader(train_ds, batch_size=16, shuffle=True)
test_loader  = DataLoader(test_ds,  batch_size=16, shuffle=False)
```

`TensorDataset` 으로 (입력, 레이블) 쌍을 묶고,
`DataLoader` 로 배치 단위 순회

9. 손실 함수 정의 & 옵티마이저 설정

```
from torch import nn, optim

n_vocab      = len(token_to_id)                 # 어휘 사전 크기
hidden_dim   = 64                               # RNN/LSTM 은닉 크기
embedding_dim = 128                             # 임베딩 벡터 차원
n_layers     = 2                                # 순환 계층 수

device      = "cuda" if torch.cuda.is_available() else "cpu"
model       = SentenceClassifier(
                  n_vocab=n_vocab,
                  hidden_dim=hidden_dim,
                  embedding_dim=embedding_dim,
                  n_layers=n_layers
              ).to(device)

criterion  = nn.BCEWithLogitsLoss().to(device)  # 이진 분류용 손실
optimizer  = optim.RMSprop(model.parameters(), lr=1e-3)
```

`BCEWithLogitsLoss` : 내부에 시그모이드+이진 크로스엔트로피 결합
`RMSprop` : 기울기 누적 없이 지수이동평균(EWMA)으로 학습률 조정

10. 모델 학습 및 테스트
    학습 모드 `model.train()` → 드롭아웃 활성화
    평가 모드 `model.eval()` → 드롭아웃 비활성화, torch.no_grad() 사용
    로짓 → sigmoid → 확률 → >0.5 → 이진 예측
    손실 감소 & 정확도 개선으로 모델 학습 상태 모니터링

11. 학습된 임베딩 계층에서 벡터 추출

```
# 학습 후 임베딩 가중치(행렬) 가져오기
embedding_matrix = classifier.embedding.weight.detach().cpu().numpy()
# 단어 ↔ 임베딩 매핑
token_to_embedding = {
    word: embedding_matrix[idx]
    for idx, word in enumerate(vocab)
}
# 예시: 1,000번째 단어의 임베딩 보기
token = vocab[1000]
print(token, token_to_embedding[token])
```

문장 분류 모델 학습 과정에서 임베딩 계층도 함께 최적화됨
해당 계층 가중치를 단어별 벡터로 사용할 수 있음

12. Word2Vec 등 사전 학습 임베딩으로 초기화
13. 사전 학습된 임베딩 계층 적용

```
class SentenceClassifier(nn.Module):
    def __init__(..., pretrained_embedding=None):
        …
        if pretrained_embedding is not None:
            # 사전 학습 임베딩 적용
            self.embedding = nn.Embedding.from_pretrained(
                torch.tensor(pretrained_embedding, dtype=torch.float32)
            )
        else:
            # 기본 임베딩 레이어 생성
            self.embedding = nn.Embedding(
                num_embeddings=n_vocab,
                embedding_dim=embedding_dim,
                padding_idx=0
            )
        …
```

14. 사전 학습 임베딩 적용 모델 학습

```
classifier = SentenceClassifier(
    n_vocab=n_vocab,
    hidden_dim=hidden_dim,
    embedding_dim=embedding_dim,
    n_layers=n_layers,
    pretrained_embedding=init_embeddings
).to(device)
# 이후 criterion, optimizer, train/test 루프 동일
```

- 사전 학습 임베딩 적용 시에도 손실·최적화 설정, 학습 절차는 동일
- 충분한 학습 데이터가 있다면 랜덤 임베딩 학습이 더 나을 수도 있음

→ 사용하는 임베딩은 언어·도메인 특성에 맞춰 선택하는 것이 중요

합성곱 신경망 (CNN)

: 이미지 인식에 특화된 인공신경망 구조

입력 데이터의 국소적인 패턴(ex. 엣지·질감)을 효과적으로 추출하기 위해 합성곱(Convolution) 연산 사용.
최근에는 문장 분류·자연어 처리에도 널리 활용.

주요 특징

- 지역 특징(Local Features)
  작은 영역(커널,필터)을 순회하며 입력값의 분포나 변화량을 계산.
  국소 패턴을 잡아내는 데 강점.

- 전역 특징(Global Features)
  여러 합성곱 계층을 쌓고 풀링(pooling)·전결합(Dense) 계층을 거치면서 전반적(글로벌) 맥락 정보 학습.

- 병렬 처리 & 가변 길이
  CNN은 입력 크기에 제한이 없고 시퀀스 순서 제약이 없어 병렬 연산이 용이.
  텍스트 길이가 길어져도 병목 없이 처리 가능.

합성곱 계층

입력 데이터에 필터(커널)를 적용해
국소 영역의 특징 맵(feature map)을 생성하는 계층

특징

- 지역 패턴 추출
  작은 필터(ex. 3×3, 5×5)를 입력 전역에 슬라이딩하면서
  국소적 변화량(엣지·질감 등)을 계산
  이미지나 음성 같은 고차원 시퀀스의 국소 패턴을 효과적으로 인식

- 가중치 공유
  입력의 모든 위치에서 동일한 필터를 사용 → 배워야 할 파라미터 수 감소
  → 과적합 방지
  동일한 특징(ex. 경계선, 특정 모양)을 입력의 어디서든 동일하게 검출

- 계층적 특징 학습
  여러 합성곱 계층을 연속으로 쌓을수록 저수준(local) 패턴부터 고수준(global) 패턴까지 점차 복합적인 특징 학습 가능
  깊이와 필터 수를 조절해 모델 용량(복잡도)을 유연하게 확장

필터

: 일반적으로 3×3, 5×5 등 작은 정방형 행렬
가중치(학습 가능한 파라미터)로 초기화 → 학습 과정에서 최적화

• 슬라이딩 윈도우 연산
입력 이미지(또는 피처 맵) 위에서 필터를 한 칸씩 이동(스트라이드=1 기준)

각 위치마다 필터와 입력의 동일 위치 요소별 곱셈
→ 그 결과를 모두 더해 단일 값으로 합산

• 특징 맵(feature map) 생성
필터 하나당 하나의 특징 맵이 생성
입력 전체를 훑으며 계산된 합성곱 값들이 특징 맵의 각 픽셀에 대응
ex. 4×4 입력에 3×3 필터 → 2×2 크기의 특징 맵

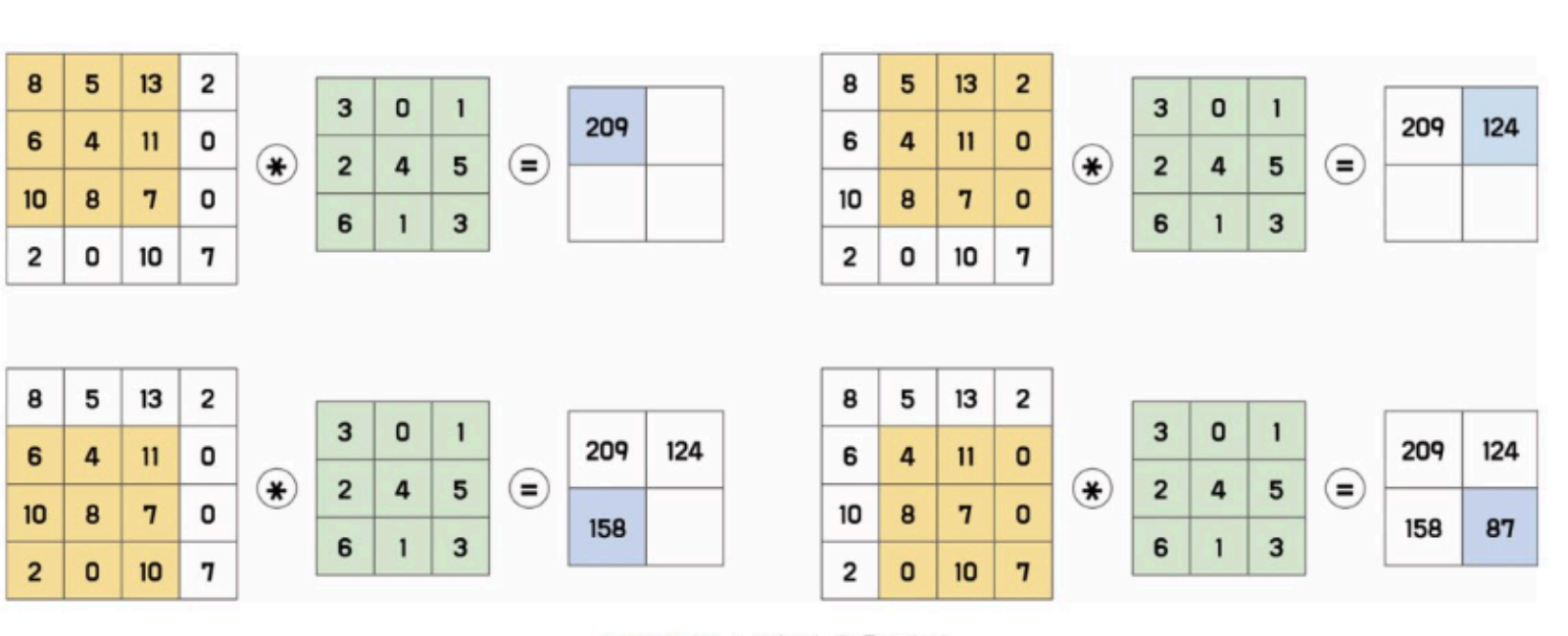


그림 6.24 특징 맵 추출 과정

• 가중치 공유 & 위치 불변성
같은 필터(가중치)를 입력의 모든 위치에 적용 → 파라미터 수 절감
특정 패턴(예: 수직 에지, 코너 등)을 입력 어디에서나 동일하게 인식

패딩

• 패딩의 필요성

합성곱 연산 시 출력 특징맵 크기가 입력보다 작아지는 문제(공간 축소)
→ 경계 근처 픽셀 정보가 적게 반영되는 현상
여러 계층 쌓을 때 지나치게 특성 맵이 작아지거나 경계 정보 손실 발생

• 제로 패딩
입력 특성 맵(또는 이미지) 가장자리에 0을 덧붙이는 기법

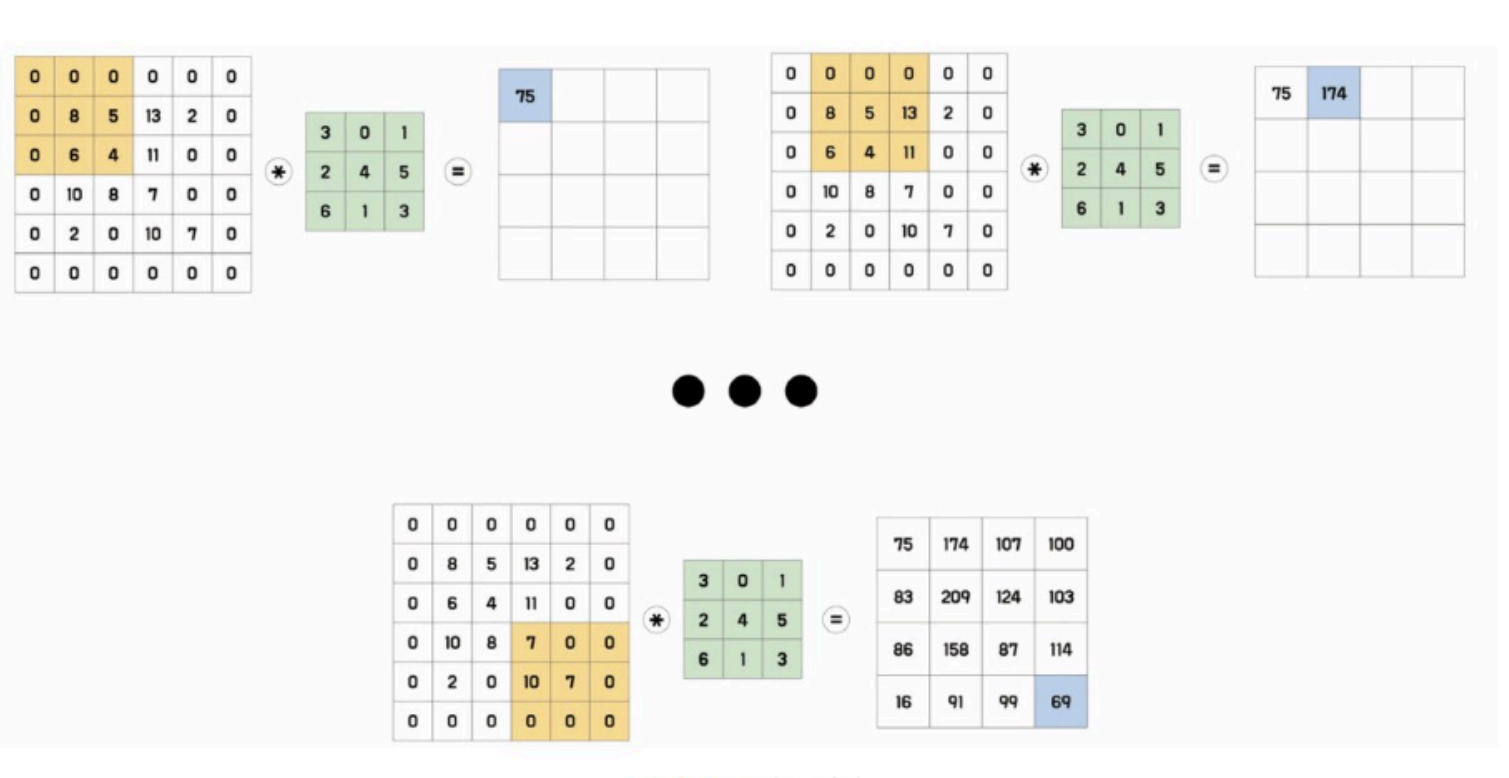


그림 6.25 제로 패딩

- 출력 크기 유지
  4×4 입력에 1픽셀 패딩 → 6×6 → 3×3 필터로 연산해도 다시 4×4 출력

- 경계 정보 보존
  원래 경계 픽셀이 필터에 적게 기여했던 것을 완화
  경계 영역의 특징도 균등하게 학습

간격 (Stride)

: 필터(커널)가 한 번에 이동하는 픽셀 간격
Stride=1이면 한 칸씩, Stride>1이면 여러 칸씩 건너뛰며 연산

• 출력 크기 조절

- Stride=1 → 출력 크기 유지(특징맵 크기가 입력과 거의 같음)
- Stride>1 → 출력 크기 감소(건너뛴 만큼 다운샘플링 효과)

• 공간 정보 보존 vs. 축소

- 작은 Stride → 더 많은 위치에서 합성곱, 공간적 세부 정보 보존
- 큰 Stride → 연산량·특징맵 크기 감소, 공간 정보 일부 희생

채널

: 입력 데이터와 필터 간의 합성곱 연산을 채널 단위로 수행
3차원 입력(RGB 이미지 등)에서 같은 위치의 값들끼리 매칭해 처리

• 동작 방식
입력이 RGB라면 R·G·B 각 채널마다 동일한 필터가 적용
각 채널별로 특징 맵(feature map)을 생성
결과적으로 출력 채널 수만큼의 특징 맵이 쌓임

• 출력 채널 수

- 많을수록 다양한 특징 학습 가능 → 모델의 표현력↑
- 적을수록 매개변수↓, 연산량·메모리 사용량·과대적합 위험↓

팽창 (Dilation)

: 합성곱 연산 시 필터와 입력 사이에 간격(stride 내 간격과 별도) 을 두어, 더 넓은 영역을 훑도록 만드는 기법

• 동작 원리
팽창값(dilation rate) 만큼 필터 요소들 사이에 빈 칸을 띄움

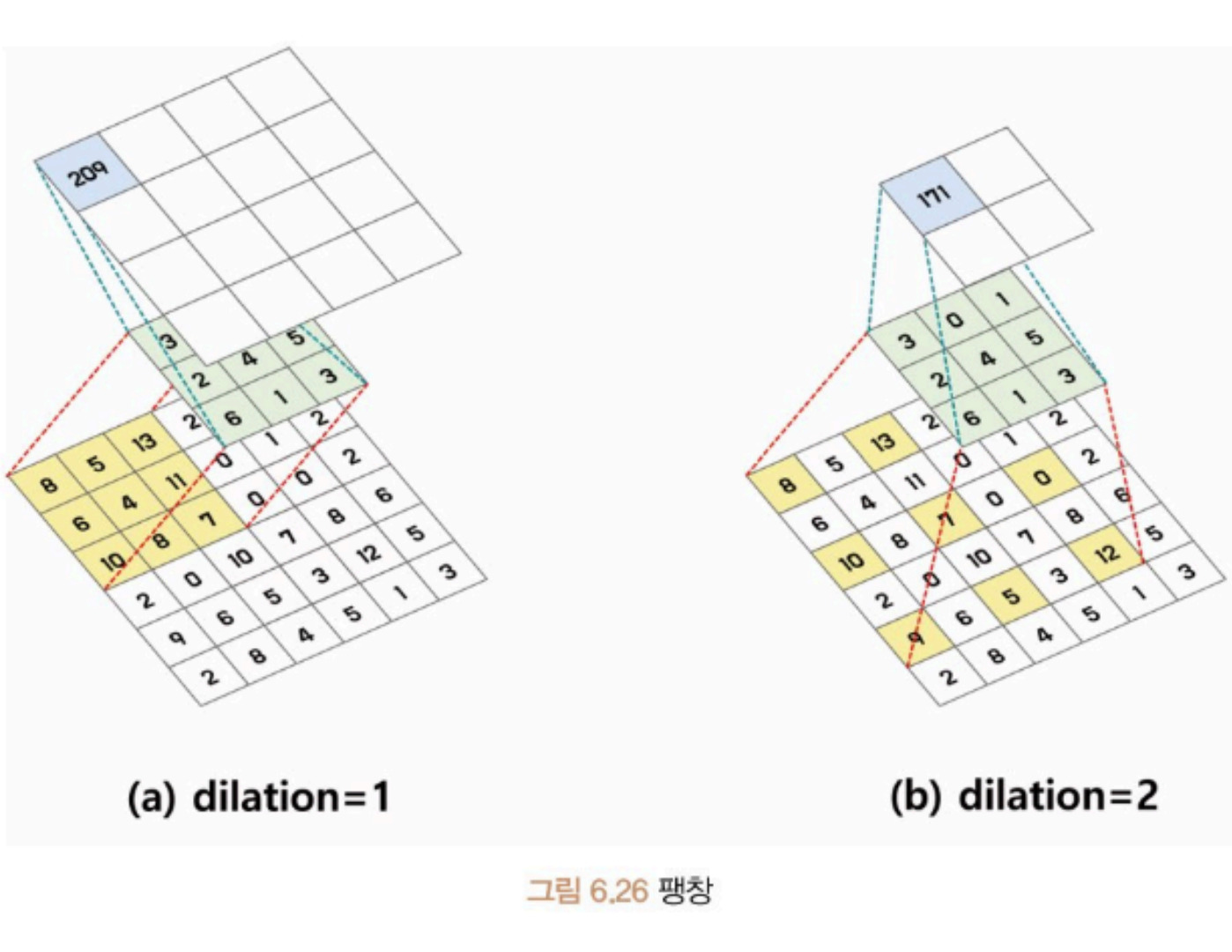


그림 6.26 팽창

- dilation=1 → 일반 합성곱(필터 요소가 연속적으로 곱해짐)
- dilation=2 → 필터 요소 사이에 한 픽셀씩 건너뜀
  → 입력의 더 넓은 영역 반영

장점

- 수용 영역(receptive field) 확장
  필터 크기를 키우지 않고도 한 번의 합성곱으로
  더 멀리 떨어진 픽셀까지 정보 취합.

단점

- 너무 큰 dilation
  불연속 간격이 너무 벌어져 주변 정보 반영이 부정확해질 수 있음.
  출력 해상도 유지하려 패딩량을 늘리면 연산량 증가.

합성곱 계층 클래스

• 2D 합성곱 계층

```
conv = torch.nn.Conv2d(
    in_channels,      # 입력 채널 수 (e.g. RGB=3)
    out_channels,     # 출력(특징) 채널 수
    kernel_size,      # 필터 크기 (정수 혹은 (h, w))
    stride=1,         # 필터 이동폭 (한 칸씩이 기본)
    padding=0,        # 입력 주변에 덧붙일 0의 개수
    dilation=1,       # 필터 요소 간격 (팽창률)
    groups=1,         # 채널 그룹 수 (그룹 합성곱)
    bias=True,        # 편향(bias) 사용 여부
    padding_mode="zeros"
                      # 패딩 방식: "zeros"(0), "reflect"(반사), "replicate"(복제)
)
```

• 출력 크기 계산
1D로 표시했을 때, 입력 길이 L_in 에 대해 출력 길이 L_out은

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times \text{padding} - \text{dilation} \times (\text{kernel\_size} - 1) - 1}{\text{stride}} \right\rfloor + 1$$

2D의 경우 가로·세로 각각에 대해 적용하면,
최종 출력 텐서의 공간 크기 (H_{out}, W_{out})를 얻음

활성화 맵 (Activation Map)

: 합성곱 계층이 추출한 특징 맵(feature map)에
비선형 활성화 함수를 적용해 얻은 출력

합성곱(Convolution)만으로는 선형 결합이 반복되어 한정된 패턴만 학습하므로, 활성화 맵을 통해 비선형성을 부여

특징

- 비선형성 덕분에 모델이 단순 선형 조합을 넘어
  다양한 패턴·구조를 학습 가능.
- 활성화 맵을 시각화하면, 네트워크가 이미지에서 어떤 부분을 중점적으로 바라보는지 이해하는 데 도움.

풀링

합성곱 계층 다음에 적용되어 특징 맵의 공간 크기를 줄이고 계산량을 감소
중요한 특징은 유지하면서 정보 압축

최대값 풀링(Max Pooling)

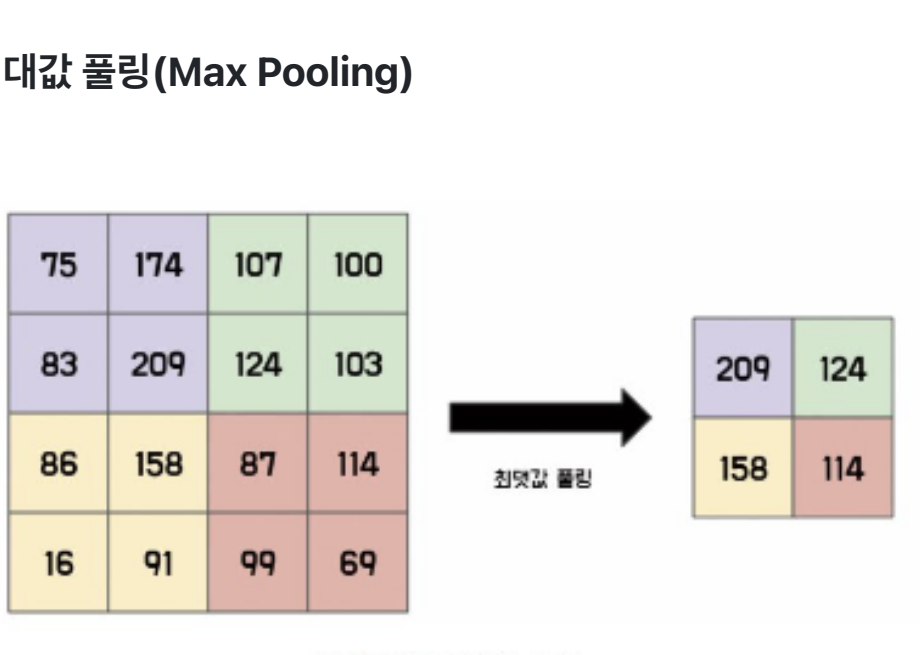


그림 6.27 최댓값 풀링

지정한 영역(window) 내에서 가장 큰 값만 선택.
강한 활성화 영역(가장 눈에 띄는 특징) 강조.

```
# 2D 최대값 풀링(Max Pooling) 클래스
# kernel_size=2: 2×2 영역 내에서
# stride=2: 창을 2칸씩 이동시키며
# padding=0: 경계 외 영역은 0으로 채우지 않음
pool_max = nn.MaxPool2d(
    kernel_size=2,  # 풀링 창 크기 지정 (예: 2×2)
    stride=2,       # 창을 몇 칸씩 띄워서 이동할지 지정
    padding=0       # 경계 처리 시 0으로 채우기 여부
)
```

평균값 풀링(Average Pooling)

window 내 값의 평균을 계산.
전체적인 특징 분포를 고르게 반영.

```
# 2D 평균값 풀링(Average Pooling) 클래스
# kernel_size=2: 2×2 영역 내의 값 평균
# stride=2: 창을 2칸씩 이동
# padding=0: 경계 외 영역은 0으로 채우지 않음
# count_include_pad=True: padding 영역(0)도 평균 계산에 포함
pool_avg = nn.AvgPool2d(
    kernel_size=2,          # 풀링 창 크기 지정 (예: 2×2)
    stride=2,               # 창을 몇 칸씩 띄워서 이동할지 지정
    padding=0,              # 경계 처리 시 0으로 채우기 여부
    count_include_pad=True # 패딩된 0 값도 평균 계산에 포함할지 여부
)
```

2D 평균값 풀링 출력 크기 공식

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times \text{padding} - \text{dilation} \times (\text{kernel\_size} - 1) - 1}{\text{stride}} \right\rfloor + 1$$

완전 연결 계층 (FC, Fully Connected Layer)

: 입력 특징(특징 맵)을 1차원 벡터로 펼친 뒤 모든 입력 노드가
모든 출력 노드와 연결되어 연산을 수행하는 계층

입력과 출력 간 가능한 모든 선형 관계를 학습하여
최종 분류·회귀 등의 결과를 출력

모델 구조

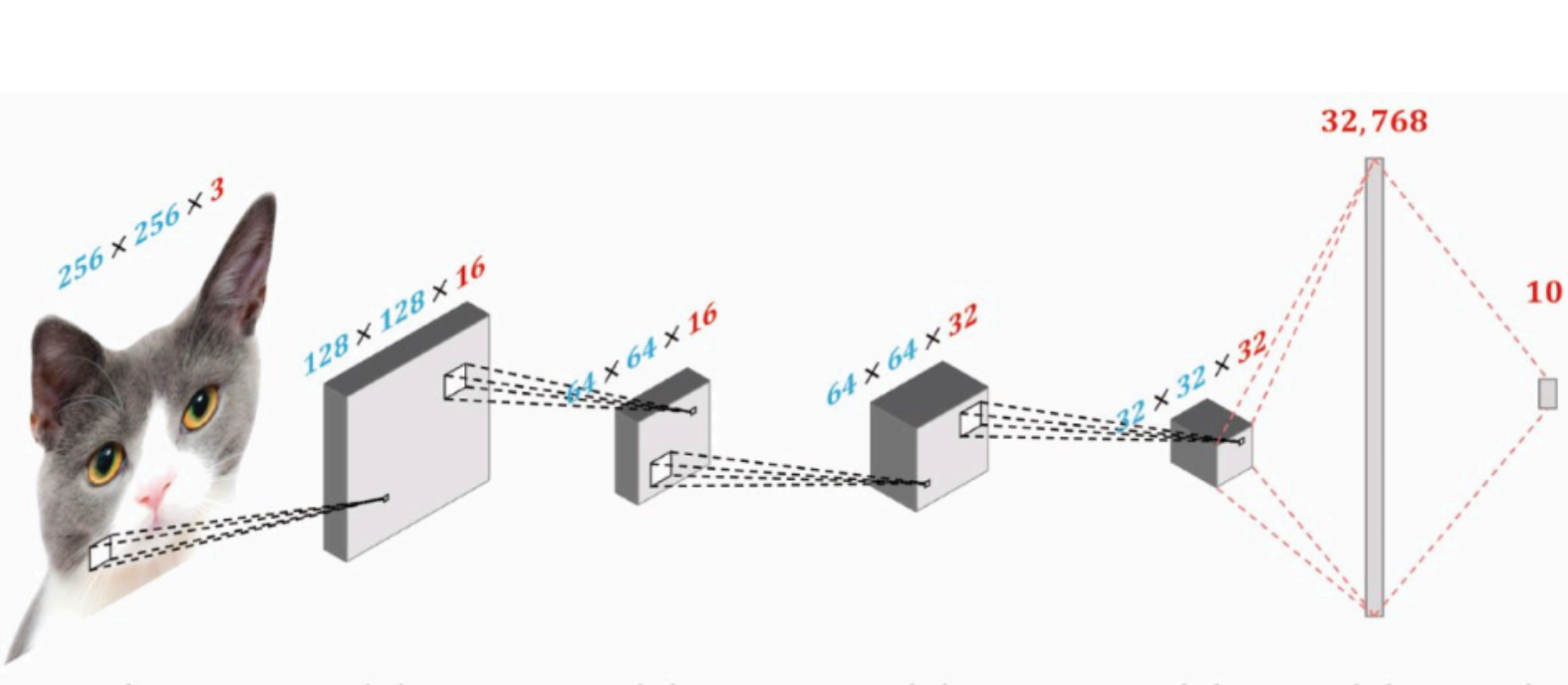


그림 6.28 합성곱 모델 구조 시각화

합성곱/풀링 계층을 거쳐 만들어진 3차원 특징 맵(높이×너비×채널 수)을
1차원 벡터로 평탄화(flatten)
→ 벡터의 길이가 곧 FC 계층의 입력 차원(input_dim)

구현 예시

```
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        # (예시) 두 번의 Conv+ReLU+Pool 후 특징 맵 생성
        self.conv1 = nn.Sequential(
            nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, stride=1, padding=1)
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        # 마지막 특징 맵 크기: (채널=32 × 높이=8 × 너비=8) 예시 → 입력 차원 = 32*8*8 = 2048
        self.fc = nn.Linear(32 * 8 * 8, 10)  # 10개 클래스로 분류

    def forward(self, x):
        x = self.conv1(x)                # → [batch, 16, H/2, W/2]
        x = self.conv2(x)                # → [batch, 32, H/4, W/4]
        x = x.flatten(start_dim=1)       # → [batch, 32*(H/4)*(W/4)]
        x = self.fc(x)                   # → [batch, 10]
        return x
```

모델 실습

텍스트 데이터의 임베딩 값은 입력 순서를 제외하면
입력값의 위치가 의미를 가지지 않음.

→ 텍스트 데이터에서 이미지 데이터처럼 2D 합성곱 필터를 사용하면
텍스트의 정보 학습 불가

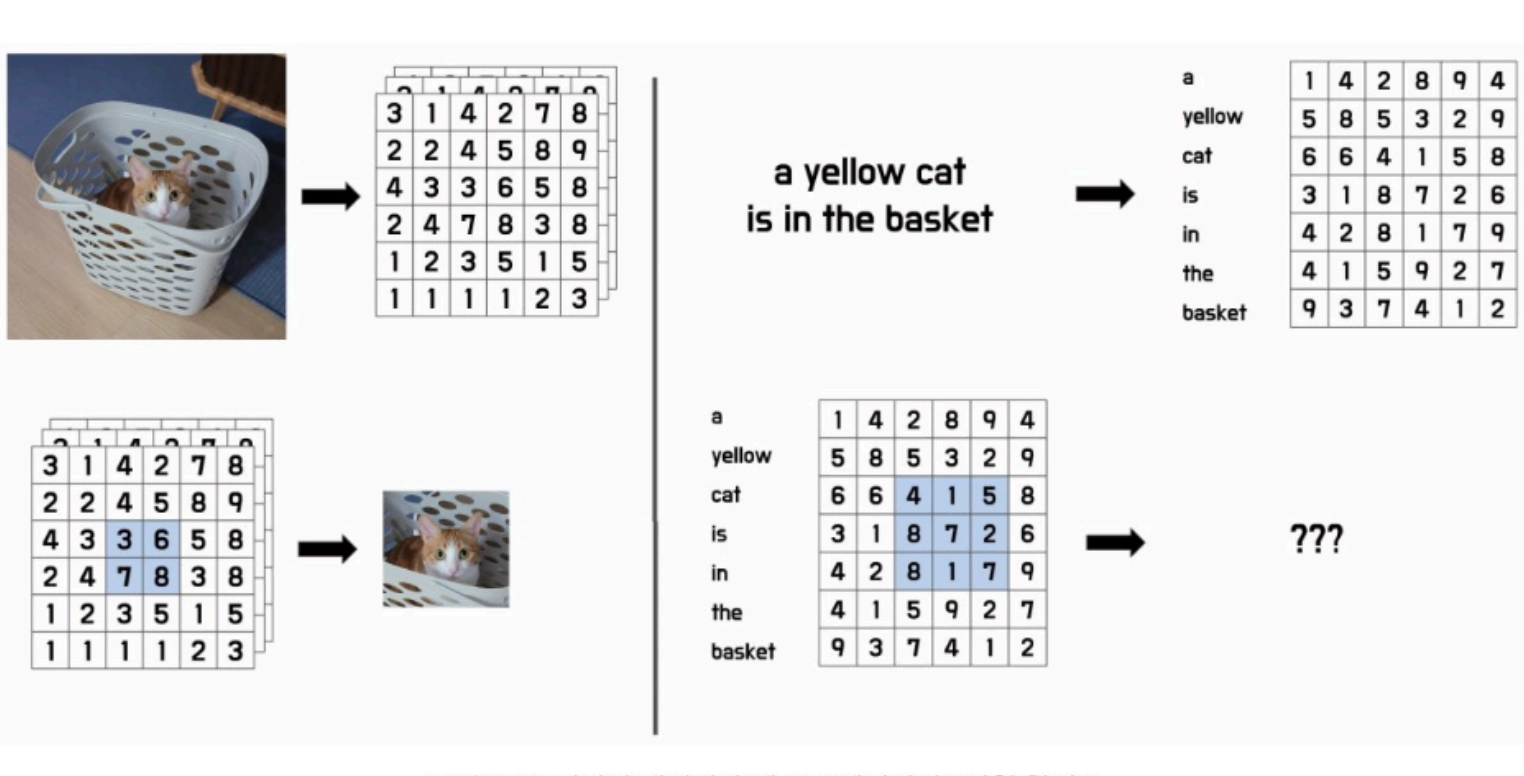


그림 6.29 이미지 데이터와 텍스트 데이터의 2차원 합성곱

텍스트 데이터는 2D 합성곱이 아닌 , 1D 합성곱을 적용해야 함.

1차원 합성곱(1D Convolution)

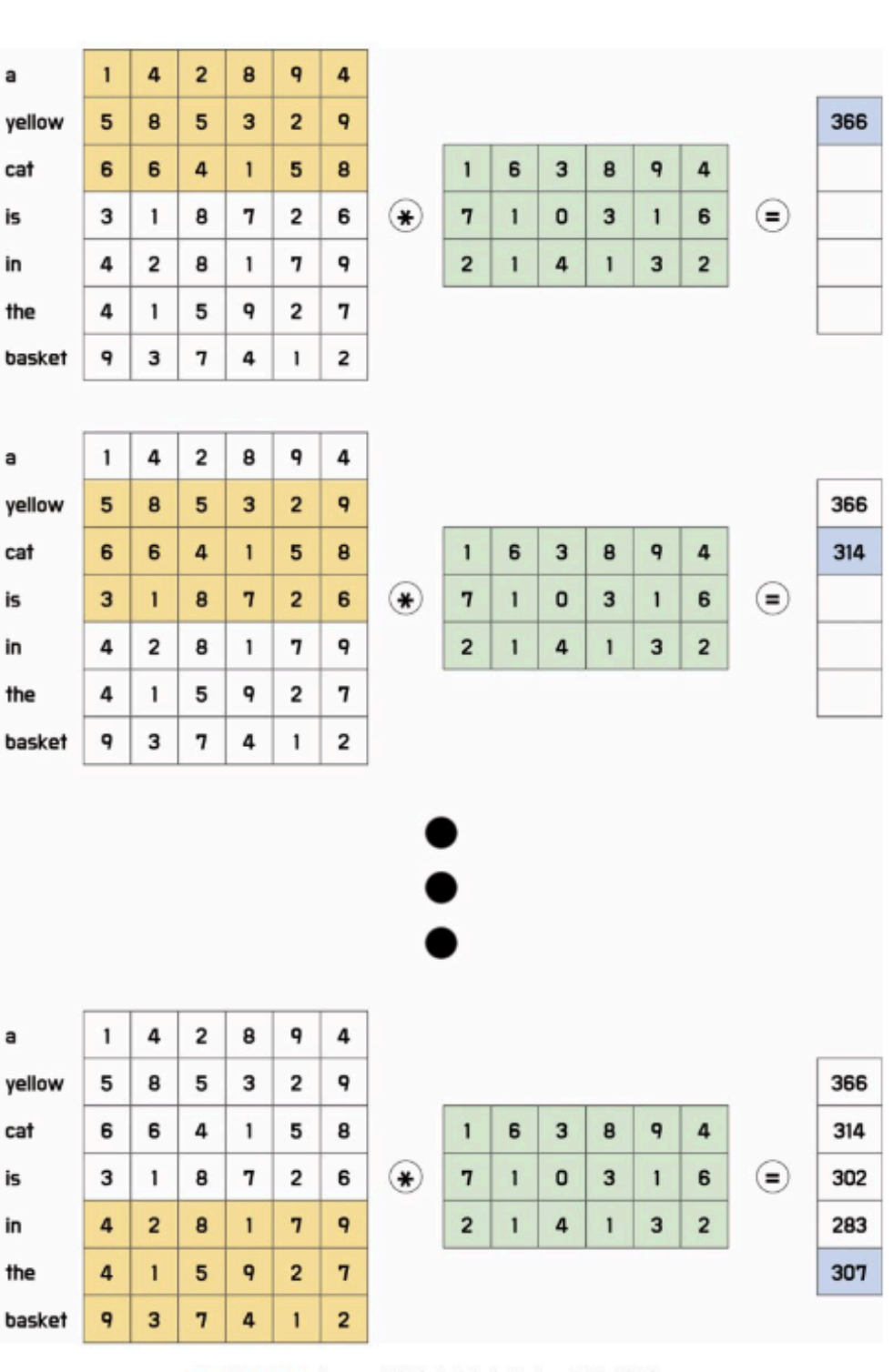


그림 6.30 텍스트 임베딩 입력값의 1차원 합성곱

• 연산 방식
필터를 시퀀스 위에서 수직(vertical) 방향으로만 움직이며(단계 = 스트라이드),
각 위치에서 입력값과 필터 원소 간 내적을 계산해 한 점의 출력을 생성

합성곱 기반 문장 분류 모델

• 전체 구조

- 임베딩층
  : 사전 학습된 임베딩(pretrained_embedding)을 nn.Embedding.from_pretrained로 초기화

- 1차원 합성곱 필터
  : 서로 다른 크기(filter_sizes 리스트)만큼의 Conv1d를 각각 하나씩 생성

- 활성화 + 풀링
  : 각 합성곱 필터 → ReLU → 남은 길이를 전부 하나로 요약하는 MaxPool1d

- 특징 벡터 결합
  : 각 필터별로 추출된 스칼라(feature)를 차원(dim=1) 방향으로 이어붙여 (torch.cat) 하나의 벡터로 만듦

- 분류기

1. `pre_classifier` : concat된 벡터 차원을 유지하는 중간 선형층
2. Dropout
3. `classifier` : 1차원 출력(긍정/부정 확률) 선형층

• 핵심 하이퍼파라미터

- `filter_sizes` : [3,4,5] 등
  → 몇 개 토큰씩 묶어 특징을 볼지(ngram 크기)
- 풀링 커널 크기 : `max_length - size + 1` 로 설정해,
  시퀀스 전체 길이에서 해당 크기를 뺀 만큼 남은 길이를 모두 압축
- `dropout` : 분류기 직전 과적합 방지 비율

• 장점

- 다양한 n-gram 학습
  : 크기별 필터로 문장 내부의 여러 범위 패턴(2-gram, 3-gram...)을
  동시에 포착
- 파라미터 공유
  : 모든 위치에 동일한 필터 적용 → 파라미터 수 절감
- 속도
  : RNN 대비 병렬 연산에 유리해 대규모 데이터에도 빠름
- 간단한 구조
  : 풀링으로 고정된 길이 벡터를 얻어 FC 분류기에 바로 연결

chelseey

이전 포스트
11. Cellular Wireless Networks

0개의 댓글

댓글을 작성하세요

댓글 작성

Powered by Stellate